

Keep It Simple: Self-Adaptive Code Graph Simplification for Accurate Vulnerability Detection

Xin Peng, Shangwen Wang, Yihao Qin, Bo Lin, Liqian Chen, Jieren Cheng, and Xiaoguang Mao

Abstract—Software vulnerability detection is crucial for high-quality software development. Recently, some studies utilizing Graph Neural Networks (GNNs) to learn the graph representation of code in vulnerability detection tasks have achieved remarkable success. However, existing graph-based approaches mainly face two limitations that prevent them from generalizing well to large code graphs: (1) the interference of noise information in the code graph; (2) the difficulty in capturing long-distance dependencies within the graph. To mitigate these problems, we propose a novel vulnerability detection method, *ANGEL*, whose novelty mainly embodies the hierarchical graph refinement and context-aware graph representation learning. The former hierarchically filters redundant information in the code graph, thereby reducing the size of the graph, while the latter collaboratively employs the Graph Transformer and GNN to learn code graph representations from both the global and local perspectives, thus capturing long-distance dependencies. Extensive experiments demonstrate promising results on three widely used benchmark datasets: our method significantly outperforms several other baselines in terms of the accuracy and F1 score. Particularly, in large code graphs, *ANGEL* achieves an improvement in accuracy of 34.27%-161.93% compared to the state-of-the-art method, *AMPLE*. Such results demonstrate the effectiveness of *ANGEL* in vulnerability detection tasks.

Index Terms—Software Vulnerability Detection, Graph Neural Networks, Graph Representation Learning.



1 INTRODUCTION

SOFTWARE vulnerabilities are specific defects and weaknesses within software components. Attackers can exploit these vulnerabilities to carry out malicious actions, endangering the lives of human societies [1]. For example, the notorious “WannaCry” ransomware exploited operating system vulnerabilities to affect hundreds of thousands of computers worldwide, causing serious economic losses and social impacts [2]. Therefore, the research and development of software vulnerability detection techniques have attracted widespread attentions from researchers [3], [4], [5], [6], [7], [8], [9].

Traditional vulnerability detection techniques primarily relied on manual auditing and expert experience, which were labor-intensive, inefficient, and required comprehensive domain knowledge [10]. In addition, there were some other studies exploring to utilize static code analysis and dynamic behavior analysis, but these were still cumbersome and prone to miss concealed vulnerabilities [11], [12], [13]. With the rapid development of deep learning technique, the automated detection of complex vulnerabilities has become a research hotspot [14], [15], [16]. These methods generally rely on large amounts of annotated data to train deep learning models, thereby improving the accuracy and

efficiency of vulnerability detection. Existing deep learning (DL)-based software vulnerability detection methods can generally be classified into two categories: token-representation-based and graph-representation-based. The token-based methods consider code as a natural language sequence, parsing the source code into a series of tokens, and then converting them into vectors for predictions [17], [18], [19]. However, unlike natural languages, source code contains rich structure and semantic information reflected by the abstract syntax tree (AST) and data/control flow relations. Therefore, in recent years, methods based on graph representation achieve more advanced detection performances [20], [21], [22], [23]. Typically, these methods use program analysis tools to construct various code structure graphs, such as control flow graph (CFG) and data flow graph (DFG), to represent complex relationships in the program [24]. Subsequently, to effectively learn the information in the graphs, Graph Neural Networks (GNNs) are widely adopted by most advanced vulnerability detectors [25]. For example, Devign [26] proposed a joint graph structure that includes AST, CFG, DFG, and natural code sequences, and used a gated graph neural network (GGNN) [27] to build a vulnerability detection model. Additionally, there are methods that use manually predefined rules to construct novel code structure graphs, thereby enriching the semantic information of the constructed graph structure [28], [29].

Although current graph-based methods exhibit promising results, one bottleneck that arises is their relatively poor performance when dealing with large code graphs. Specifically, Fig. 1 shows the distribution of node numbers in the code property graphs [32] of different vulnerable programs from the widely-used Devign [26], Reveal [30] and Big-Vul [31] datasets. We note that the number of nodes generally exhibits a long-tail distribution, and such a phenomenon can be observed from all the three datasets,

- Xin Peng, Shangwen Wang, Yihao Qin, Bo Lin, Liqian Chen, and Xiaoguang Mao are with the Key Laboratory of Software Engineering for Complex Systems, College of Computer Science and Technology, National University of Defense Technology, Changsha, China.
E-mails: xinpeng@nudt.edu.cn, wangshangwen13@nudt.edu.cn, yihao-qin@nudt.edu.cn, linbo19@nudt.edu.cn, lqchen@nudt.edu.cn, and xg-mao@nudt.edu.cn
- Jieren Cheng is with the Hainan University, Haikou, China.
E-mails: cjr22@163.com
- Shangwen Wang is the corresponding author.

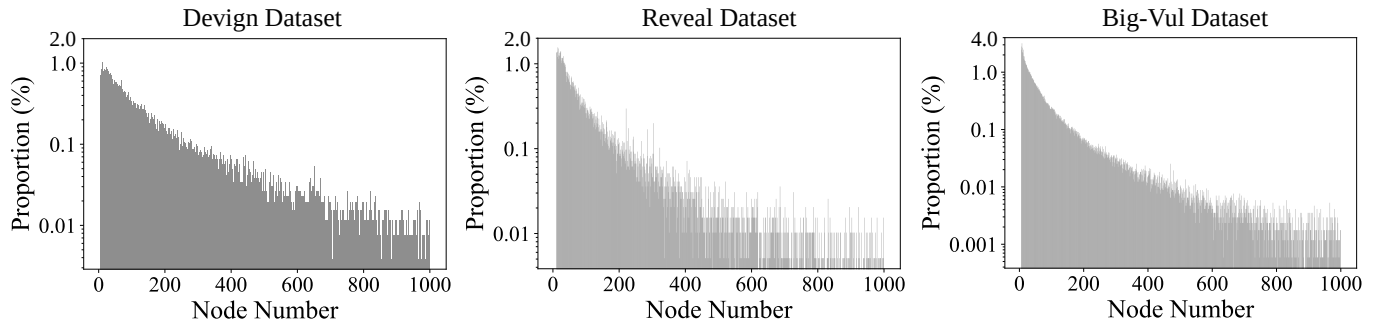


Fig. 1: Statistics on the distribution of graph sizes in the Devign [26], Reveal [30] and Big-Vul [31] datasets.

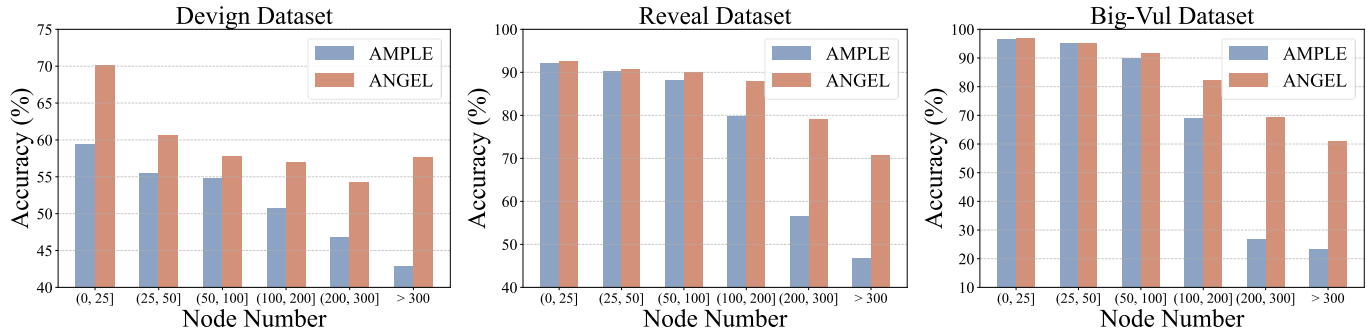


Fig. 2: Accuracy with different number of nodes in the Devign [26], Reveal [30] and Big-Vul [31] datasets.

which indicates that this is a universal characteristic for real-world vulnerabilities. Although the majorities are in the (0, 100] interval, there are still considerable samples with more than 100 nodes, and they are unevenly distributed across various intervals with the largest code graph containing more than 1K nodes (due to the space constraint, the maximum value is set to 1K in the figure). This disparity in distribution forces vulnerability detectors to bias towards fitting graphs with fewer nodes, leading to a performance bottleneck where large graphs are difficult to be precisely predicted [33], [34], [35], [36]. As shown in Fig. 2, we demonstrate the accuracy of AMPLE [37], the state-of-the-art graph-based vulnerability detectors, under different node count partitions on the three datasets. Taking the Reveal dataset as an example, AMPLE achieves an accuracy of 92.25% in the (0, 25] interval, while its effectiveness significantly declines when the number of node exceeds 100 (e.g., its accuracy is only 46.85% when the number of nodes is larger than 300). Such results demonstrate that the performances of state-of-the-art vulnerability detection techniques severely decreases with the scales of code graphs increase.

Upon reviewing the working mechanisms of existing approaches, their ineffectiveness on large code graphs could mainly be attributed to the following two reasons. ① **The large amount of noisy information in the graph could affect the prediction.** Existing methods usually use global pooling after the GNN layers to convert the graph into a vector representation [26], [37], [38]. In such a process, information from all the nodes in the graph is utilized to make the final decision. However, there might be only a small number of key nodes that incur the vulnerability while the other nodes are not directly related to the vulnerability [39], [40]. In such cases, the prediction of existing approaches could be influenced by the large amount of vulnerability-irrelevant noisy

information, which correspondingly leads to inaccurate results. Although AMPLE incorporates a graph simplification technique [37], this process relies on pre-defined heuristics and typically results in the reduction of only a limited number of nodes in the graph. This limitation could potentially explain the inefficacy of AMPLE on large code graphs, as will be demonstrated in Section 3.

② **The long-distance dependencies in the graph could be neglected.** By utilizing standard GNN layers, existing methods only gather information from neighboring nodes for generating the representation of a node [30], [37], [41]. However, it is widely known that software code contains long-distance dependency relations. For instance, a variable defined at the beginning of a function might be used in the final return statement of that function. Fig. 3 displays the distribution of the longest distance between two nodes for the programs in the Devign [26], Reveal [30] and Big-Vul [31] datasets. It can be observed that more than half of the programs contain a pair of nodes in their graphs whose distance exceeds or equal to four. As shown by existing studies, there could be challenges such as the over-smoothing and over-compression for GNNs to fit graphs whose longest path is greater than three [42], which may lead to the failure of GNNs of comprehensively understanding the graphs. Consequently, critical dependency relations could be missed by existing approaches when detecting vulnerabilities in such programs.

In this paper, we propose a hierarchical and context-aware Graph Representation Learning approach that aims at mitigating the ineffectiveness of existing vulnerability detection techniques on large code graphs. To overcome the aforementioned two limitations, ANGEL contains two customized modules, i.e., an importance-based graph simplification module and a global & local feature aggregation module. After parsing the programs

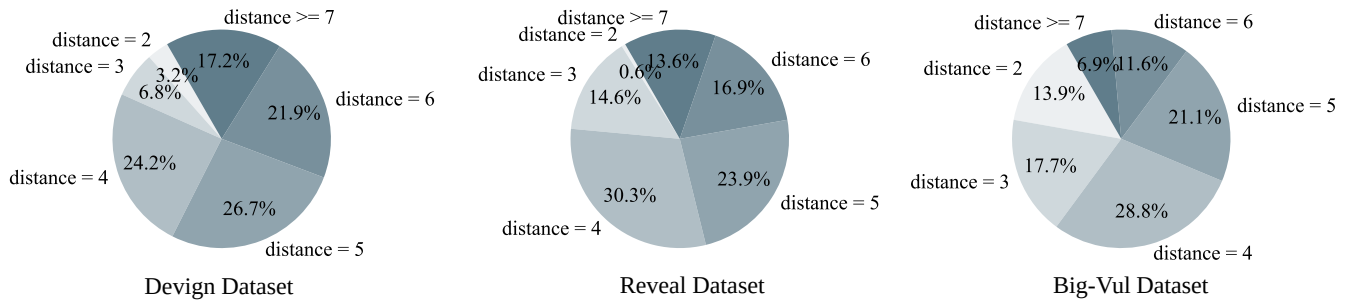


Fig. 3: Statistics of longest distance in the Devign [26], Reveal [30] and Big-Vul [31] datasets.

into graph structures, the importance-based graph simplification module hierarchically refines the graph, aiming at reducing the size of the graph so that vulnerability-irrelevant noisy information could be filtered. In contrast to AMPLE, our objective is to autonomously learn the characteristics of the pivotal nodes within the graph rather than depending on pre-established heuristics, which are recognized for their lack of flexibility. To that end, Top-k pooling [43] is adopted to select critical nodes based on their importance scores, which are calculated by a learnable matrix. Note that the simplification process is self-adaptive: for each code graph, this phase terminates when the number of left nodes becomes lower than a pre-defined threshold. This means that there might be unnecessary to perform simplifications for small-scale code graphs, while large-scale graphs may iteratively undergo several numbers of pooling operations. This strategy enables us to retain the size of the simplified graph within a reasonable range. In contrast, the existing AMPLE typically only reduces a minimal portion of nodes in the original graph (less than 50%) and thus the simplified graph can still be in large scale. The number of filtered nodes is dynamically adjusted during different pooling layers. Specifically, in the first pooling step, ANGEL only preserves 10% of the total nodes for quickly removing noisy information; while in the last several pooling layers, 50% of the input nodes are preserved for precisely identifying the critical information. Taking the simplified graph as input, we utilize GNNs for feature extraction to generate a one-dimensional vector representation. Different from existing methods, ANGEL borrows the weapon of Graph Transformer (GT) [44], [45], a network that can model the semantic information of a graph from the holistic perspective, i.e., the long-distance dependency relations could be captured by such a model. Notably, we are the first to integrate GNN and GT for the vulnerability detection task, leveraging their respective strengths to design a global & local feature aggregation module. Due to the varying scale and complexity of each code graph, the generality of the state-of-the-art method AMPLE, which rely on GNN and Convolutional Neural Networks (CNN) with fixed kernel sizes, is inherently limited. In contrast, our feature aggregation module alternates between traditional GNN and GT layers, with residual connections enhancing its learning capacity. Specifically, GNN focuses on modeling the semantics of nodes from their adjacent neighbors, while GT leverages a multi-head self-attention mechanism to extract global features from the entire graph for representation generation. In this manner, both global and local context relationships in the graph are captured, which alleviates the issue of long-range dependencies and achieves a

more accurate extraction of semantic information from the graphs.

To evaluate the effectiveness of ANGEL, we chose three commonly-used public datasets for experiments: Devign [26], Reveal [30], and Big-Vul [31]. We selected eight state-of-the-art learning-based vulnerability detection techniques as baselines for comparison, where two are based on tokens, and the remaining six are based on graphs. The experimental results demonstrate that ANGEL achieves significant improvements compared to all the baselines. Specifically, ANGEL achieves a 37.70%-202.20% improvements in terms of the F1 scores across the three datasets compared to the token-based methods, while such improvements become 11.55%-164.01% when it comes to the comparison with graph-based methods.

This paper makes the following contributions:

- **Problem.** We reveal that due to the complexity of real-world projects, current graph-based vulnerability detection models fall short on detecting vulnerabilities in large code graphs, significantly limiting their generalization ability and performance.
- **Approach.** We propose a novel vulnerability detection method named ANGEL. This approach employs a meticulously designed graph simplification module to adaptively reduce the size of code graphs, while integrating GNN and GT to enable the network to extract code graph features from both local and global perspectives.
- **Evaluations.** Experiments on three public benchmark datasets indicate that our proposed ANGEL outperforms the state-of-the-art baseline methods. Our source code is available at <https://github.com/XinPeng97/ANGEL>.

2 BACKGROUND & RELATED WORKS

2.1 Graph Neural Networks

In the software engineering domain, graphs have been used to represent various structures of code, such as CFGs, PDGs, ASTs, and others [46], [47]. These graph structures can reflect the structures of the program code, the dependencies between variables, as well as the processes of program execution [48]. Due to their superior performance, GNNs have been adopted for learning graph data and have been widely applied in tasks such as code clone detection [49] and vulnerability detection [50]. GNNs learn node representations or graph-level representations in an end-to-end manner. Specifically, each node communicates with its neighbors through a propagation operator, after which various aggregation functions (e.g., sum, average, or max) are used to integrate information from the neighboring nodes [51]. Graph Convolutional

Networks (GCN) [52] and Graph Attention Networks (GAT) [53] are classical examples of GNNs. GCN introduces the concept of convolution into graph neural networks. Based on GCN, GAT introduces an attention mechanism to provide different aggregation weights for the information from neighbor nodes. Previous graph-based vulnerability detection studies have employed several layers of traditional GNNs, such as GCN and GGNN, to capture local information in the code structure graphs.

Recently, researchers have observed the long-range dependency problem in GNNs, leading to the design of a specialized Transformer architecture for graph-structured data, known as the GT network [54]. GT dynamically computes interaction weights with all nodes through the attention mechanism, effectively capturing global information within the graph and adapting to complex graph structures. Additionally, with multi-head attention, GT can learn representations of nodes in different subspaces, enriching the feature representation of nodes. To fully capture different patterns of vulnerabilities, in this work, we combine the characteristics of GNN and GT, utilizing their complementarity to learn the code structure graphs from both global and local perspectives.

2.2 Hierarchical Graph Representation Learning

Hierarchical graph representation learning aims to mine finer-grained and important node information in graph structures through an iterative manner [55], where hierarchical pooling is usually utilized to generate representations for smaller subgraphs. Typically, hierarchical pooling can be divided into two types: clustering-based pooling and node-dropping-based pooling [56]. The former transforms hierarchical learning into a clustering problem, in which nodes in the graph are partitioned into different clusters and new nodes will be generated based on the cluster centroids. For example, DiffPool [57] learns differentiable soft cluster assignments for nodes at every layer of GNNs, mapping related nodes to a set of clusters to form the input for the next layer of the network. One notable drawback of this type of method is that it usually has high time and space complexity. The latter, i.e., the node-dropping-based pooling, designs a learning scoring function to remove nodes with relatively low scores, thereby obtaining subgraphs composed by important nodes. For instance, Graph U-Nets [43] designs a similarity-based pooling operator to progressively learn fine-grained subgraphs, while GXN [58] uses a local pooling operator designed based on maximizing mutual information. Typically, in the code graphs, not all the nodes are directly related to the vulnerability. That is to say, a number of nodes could be unrelated to the vulnerability and may bring noisy for the detection results [40]. Therefore, in this work, we exploit hierarchical graph learning methods to retain important nodes in the code graphs based on node dropping, thereby filtering irrelevant information.

2.3 DL-based Vulnerability Detection

Learning-based approaches have been extensively applied to the task of vulnerability detection [59], [60], [61], [62]. Previous works have focused on representing source code as natural language sequences and applying sequence models to learn vector representations of vulnerability information [63], [64], [65]. Generally speaking, source code contains complex syntactic and semantic structures, which is why graph-based methods have achieved state-of-the-art performance [37]. These methods typically use static analysis to transform the source code into a graph,

```

Case 1
1. uint8_t* FAST_FUNC udhcp_get_option32(struct dhcp_packet *packet, int code)
2. {
3.     uint8_t *r = udhcp_get_option(packet, code);
4.     if (r) {
5.         - if (r[-1] != 4)
6.         + if (r[-OPT_DATA + OPT_LEN] != 4)
7.             r = NULL;
8.     }
9.     return r;
}

Case 2
1. static char **fill_envp(struct dhcp_packet *packet)
2. {
3.     int envc;
4.     int i;
5.     char **envp, **curr;
6.     const char *opt_name;
7.     uint8_t *temp;
8.     .....
99. - if (code == DHCP_SUBNET) {
100. + if (code == DHCP_SUBNET && temp[-OPT_DATA + OPT_LEN] == 4) {
101.     /* Subnet option: make things like "$ip/$mask" possible */
102.     uint32_t subnet;
103.     move_from_unaligned32(subnet, temp);
104.     *curr = xasprintf("mask=%u", mton(subnet));
105.     putenv(*curr++);
106.     .....
133.     return envp;
134. }

```

Fig. 4: Two different functions from the busybox project have the same type of vulnerability (CWE-125). The red-colored code is the vulnerable code, and the green-colored code is the repaired code.

and employ some form of word embedding to generate the initial attributes for nodes [15]. Reveal [30] utilizes a CPG to represent source code information, then employs a GGNN to capture local structural information in the code. DeepDFA [66] designs a CFG with abstract dataflow embeddings to capture information such as API calls and operations, and then uses a GGNN to learn complex structural relations. Furthermore, some approaches design different network architectures based on various code representations to learn code representations. For example, Vul-SPG [67] proposes a sliced property graph and designs a triplet attention mechanism to enhance the capability of aggregating node information, thereby achieving better detection performance. Similarly, MAGNET [29] views the code structure as a heterogeneous graph and uses a graph neural network with an attention mechanism to learn vulnerability patterns.

Nevertheless, these techniques can frequently fail when confronted with large code graphs and long-range dependencies [37]. This is primarily because they indiscriminately assimilate knowledge from diverse source codes using conventional GNNs, which are ill-equipped to effectively address these specific challenges. Our approach, while still grounded in graph structures, incorporates tailored solutions to tackle the two challenges, as will be detailed in Section 4.

3 MOTIVATING EXAMPLE

To demonstrate that the state-of-the-art graph-based vulnerability detection methods perform comparatively worse on large graphs, we select two detailed cases from the Big-Vul dataset as the example, both of which are with the out-of-bounds read vulnerability type (CWE-125). These two examples are shown in Fig. 4, where brief excerpts of two different functions from the busybox project are presented.¹ These two examples have the same vulnerability type and developers even leave the same comment during the vulnerability fixing activities: “when decoding DHCP_SUBNET,

1. <https://git.busybox.net/busybox/>

ensure it is 4 bytes long". Nonetheless, one significant difference between these two functions is their code length. Specifically, Case 1 contains only 9 lines of code, while Case 2 possesses up to 134 lines of code. As a result, when Case 2 is transformed into the code graph, it would contain a large number of nodes irrelevant to the vulnerability, which could influence the prediction of the state-of-the-art detection approaches and thus cast side effects on the detection result.² In fact, the state-of-the-art vulnerability detection approaches can indeed be misled by the redundancy information in the code graph. Taking the state-of-the-art approach AMPLE [37] as an example, it correctly detects the first case as a vulnerable function, however, it incorrectly predicts the second case as non-vulnerable.

We also recall that in the AMPLE study, the authors showcased AMPLE as the most adept existing method for managing large code graphs. Intuitively, if AMPLE encounters challenges in making accurate predictions, it is plausible that other state-of-the-art approaches may also encounter similar difficulties. Indeed, we have confirmed that other state-of-the-art approaches like Devign and Reveal also fail to predict the vulnerability in Case 2.

Besides the noisy information in the code graph, another challenge brought by the large-scale code snippet is the long-range dependencies in the graph which may not be easily captured by the existing approaches. Taking Case 2 as an example, the array related to the vulnerability, i.e., `temp`, is first initialized in line 7, however, the buggy line that needs to check the content of `temp` occurs in line 99. Between these two statements, `temp` undergoes two assignments, two conditional expressions where it is used to judge if the condition is satisfied, and it is also used as an argument for one time. Therefore, it is quite hard for existing approaches to comprehensively comprehend the data and control dependency relations of the array `temp`, and thus fail to recognize that the buggy line also needs to check the content of `temp`.

From the above examples, we find that state-of-the-art vulnerability detection approaches perform relatively poor on large code graphs, probably due to the following two reasons: (1) **they could be misled by the vulnerability-irrelevant information in the large graphs**, and (2) **they fall short on understanding the long-range dependencies in the large graphs**. Therefore, we are motivated to design a vulnerability detection method by designing strategies to address these two limitations respectively, thus making it able to generalize well to large code graphs.

4 APPROACH

4.1 Overview

In this work, we propose a vulnerability detection approach ANGEL, aiming at addressing the challenges brought by large code graphs. The key novelty of ANGEL lies in its (1) self-adaptive learning of important nodes in the graph to hierarchically reduce the graph size, and (2) collaborative utilization of GNN and GT to aggregate the features from both local and global contexts. Generally, ANGEL works in a four-stage paradigm that consists of parsing, hierarchical graph refinement, context-aware graph encoding, and classification. In the following, we first provide some basic definitions and then introduce each stage briefly.

². In this paper, vulnerability-relevant information denotes those that have data/control dependency with the variables from the vulnerable statements, while other entities in the code are denoted as vulnerability-irrelevant information.

```

1. int diAlloc(struct inode *pip, bool dir, struct inode *ip)
2. {
3.     int rc, ino, iagno, addext, extno, bitno, sword;
4.     int nwords, rem, i, agno, dn_numag;
5.     u32 mask, inosmap, extsmap;
6.     struct inode *ipimap;
7.     struct metapage *mp;
8.     ino_t inum;
9.     struct iag *iagp;
10.    struct inomap *imap;
11.    ipimap = JFS_SBI(pip->i_sb)->ipimap;
12.    imap = JFS_IP(ipimap)->i_imap;
13.    JFS_IP(ip)->ipimap = ipimap;
14.    JFS_IP(ip)->fileset = FILESYSTEM_I;
15.    if (dir) {
16.        agno = dbNextAG(JFS_SBI(pip->i_sb)->ipbmap);
17.        AG_LOCK(imap, agno);
18.        goto tryag;
19.    }
20.    agno = BLKTOAG(JFS_IP(pip)->agstart, JFS_SBI(pip->i_sb));
21.    dn_numag = JFS_SBI(pip->i_sb)->bmap->db_numag;
22.    if (agno < 0 || agno > dn_numag)
23.        return -EIO;
24.    if (atomic_read(&JFS_SBI(pip->i_sb)->bmap->db_active[agno])) {
25.        agno = dbNextAG(JFS_SBI(pip->i_sb)->ipbmap);
26.        AG_LOCK(imap, agno);
27.        goto tryag;
28.    }
29.    inum = pip->i_ino + 1;
30.    ino = inum & (INOSPERIAG - 1);
31.    if (ino == 0)
32.        inum = pip->i_ino;
33.    AG_LOCK(imap, agno);
34.    IREAD_LOCK(ipimap, RDWRLOCK_IMAP);
35.    iagno = INOTOIAG(inum);
36.    if ((rc = diIAGRead(imap, iagno, &mp))) {
37.        IREAD_UNLOCK(ipimap);
38.        AG_UNLOCK(imap, agno);
39.        return (rc);
40.    }
41.    .....
133.    return (diAllocAny(imap, agno, dir, ip));
134. }

```

Fig. 5: An example of a vulnerability in the Linux kernel.

4.1.1 Symbol and Task Definition

Given a set of graphs $\mathcal{G} = \{G_1, G_2, \dots, G_M\}$ associated with the corresponding labels $\mathcal{Y} = \{y_1, y_2, \dots, y_M\}$ and node numbers $\{N_1, N_2, \dots, N_M\}$. For each graph $G_i = \{\mathcal{V}^{G_i}, \mathcal{E}^{G_i}, \mathbf{X}^{G_i}\}$, the corresponding adjacency matrix is $\mathbf{A} \in \mathbb{R}^{N \times N}$, and the degree matrix is $\mathbf{D} \in \mathbb{R}^{N \times N}$. Here, $\mathcal{V}$ represents the set of nodes, $\mathcal{E}$ represents the set of edges, and $\mathbf{X} \in \mathbb{R}^{N \times d}$ denotes the raw feature matrix. The symmetric normalized adjacency matrix $\tilde{\mathbf{A}} \in \mathbb{R}^{N \times N}$ is calculated through $\mathbf{D}^{-\frac{1}{2}}(\mathbf{A} + \mathbf{I})\mathbf{D}^{-\frac{1}{2}}$, where $\mathbf{I} \in \mathbb{R}^{N \times N}$ is an identity matrix.

Given a vulnerable sample F_v , it typically consists of the vulnerability lines f_l , which explain how the vulnerability arises; the contextual statements f_c , which are associated with the vulnerability; and a significant number of statements f_n that are not directly related to the vulnerability. The sample f_v is formally defined as:

$$F_v = f_l \cup f_c \cup f_n \quad (1)$$

Here, f_l represents the core characteristics that expose the vulnerability, f_c aids in understanding the conditions under which the vulnerability emerges, and f_n may act as distracting information in the vulnerability detection task. We employ the analysis tool Joern³ to transform F_v into a code property graph $G_v = \{\mathcal{V}, \mathcal{E}, \mathbf{X}\}$. Accordingly, the sets of nodes corresponding to the code lines f_l , f_c , and f_n are defined as $\mathcal{V}_l$, $\mathcal{V}_c$, and $\mathcal{V}_n$, respectively. Next, the edges present in the AST, CFG, and PDG are defined as $\mathcal{E}$, which connect all nodes from the node sets $\mathcal{V}_l$, $\mathcal{V}_c$, and $\mathcal{V}_n$. Each node in the set $\mathcal{V}$ possesses foundational information (such as node type and source code snippet) to represent the semantics of the node. We encode this information as vectors, which serve as the feature matrix $\mathbf{X}$ in the code property graph G_v . The

3. <https://joern.io/>

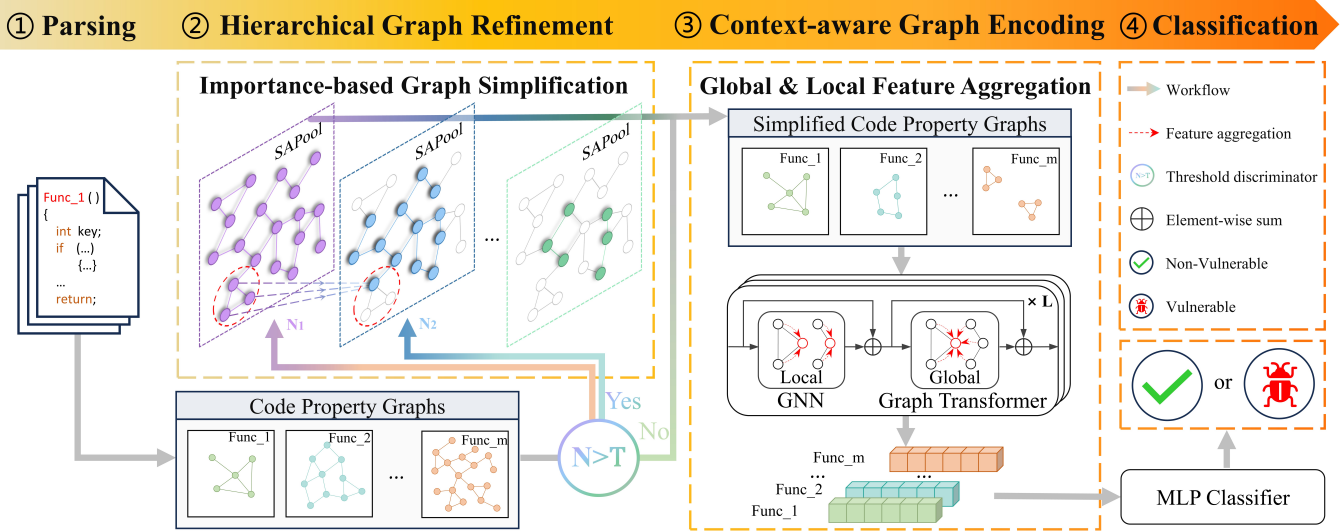


Fig. 6: The architecture of ANGEL, which mainly contains two core module: (1) importance-based graph simplification module; (2) global & local feature aggregation module. The four-stage paradigm (i.e., parsing, hierarchical graph refinement, context-aware graph encoding, and classification) endows ANGEL with enhanced effectiveness for vulnerability detection.

attribute characteristics of each node can be formally expressed as x . Since focusing on the vulnerable lines f_l and the associated contextual statements f_c is essential for uncovering the root causes of vulnerabilities, we regard the corresponding nodes $\mathcal{V}_l$ and $\mathcal{V}_c$ as **key nodes**. The structural relationships between and within these statements are represented by the edges $\mathcal{E}_p$ in the code graph. Accordingly, the attribute information x contained within each node of $\mathcal{V}_l$ and $\mathcal{V}_c$, along with the edges $\mathcal{E}_p$ connecting these nodes, together form the **characteristics of key nodes** in the code graph G_v . We use the real-world vulnerability CVE-2023-52805⁴ as an example. The green code snippet in Fig. 5 demonstrates the patch for this vulnerability, with red backgrounds indicating the vulnerable lines of code, and orange backgrounds representing the associated statements related to the vulnerability. In the unpatched code, the absence of boundary checks for the value of the variable `agno` leads to the possibility of out-of-range values. To detect this vulnerability, it is crucial to focus on the statements involving the variables `agno` and `dn_numag` (such as those related to computation logic and boundary checks). In this case, the nodes corresponding to the lines of code with red backgrounds are defined as $\mathcal{V}_l$, and those with orange backgrounds as $\mathcal{V}_c$. The $\mathcal{V}_l$ and $\mathcal{V}_c$ nodes are considered pivotal nodes, as they carry the contextual information related to the vulnerabilities. The dependency relationships between these statements are implemented through the PDG, defined as $\mathcal{E}_p$ (i.e., the green arrows in Fig. 5). By concentrating on these nodes, the learning-based model effectively captures essential patterns associated with potential vulnerabilities in the code, thereby significantly improving its vulnerability detection capabilities [68].

For the task of vulnerability detection, we aim to train a network that maps each code graph $G_i \in \mathcal{G}$ to a low-dimensional vector, and then use a classifier to perform binary classification.

4.1.2 Pipeline

The overall architecture of ANGEL is shown in Fig. 6, with its learning paradigm divided into four stages:

- **Parsing:** Given the source code of a function, we utilize static analysis tools to extract the code property graph of the function, which serves as the input to the following neural network.
- **Hierarchical Graph Refinement:** This stage aims at reducing the complexity of the input graph, for which we propose an importance-based graph simplification module composed of several pooling layers (referred to as SAPool), each of which filtering a certain percentage of nodes from the input graph. Generally, according to the size of the code property graph, the inputs are self-adaptively fed into different numbers of SAPool layers to obtain the simplified code property graph.
- **Context-aware Graph Encoding:** This stage aims at encoding the simplified graph and obtaining a vector representation for the final classification. To that end, we designed the global & local feature aggregation module by integrating GNN with GT. Specifically, we design a cascading GNN and GT with residual connections as the backbone network, which encodes the refined code property graph from both global and local perspectives to obtain the graph representation. Subsequently, we employ global average pooling to embed the graph information into a vector.
- **Classification:** In the final stage, we feed the graph embedding vector into a Multiple Layer Perception (MLP) layer to obtain an output probability, which denotes the prediction result, and train the network with the cross-entropy loss function.

Finally, during inference, given the source code of a function, ANGEL can perform binary classification (vulnerable and non-vulnerable) in an end-to-end manner with the above well-trained models. Please note that the two critical modules, i.e., the

4. <https://nvd.nist.gov/vuln/detail/cve-2023-52805>

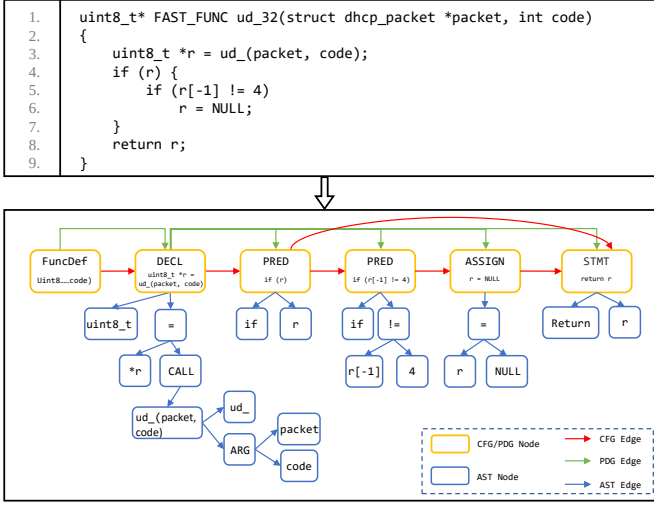


Fig. 7: A CPG for the example source code.

hierarchical graph refinement and context-aware graph encoding are originally proposed in this study.

In the following subsections, we will detail the technical specifics of each stage.

4.2 Parsing

Code contains rich structural information, and using graphs can better represent the non-linear structure within the code. Similar to previous methods, ANGEL utilizes the Joern tool to parse the source code, thereby generating a code property graph. The generated code property graph includes three important structures: AST, CFG, and PDG, which are rich in semantic and syntactic information. Specifically, given a source code file of a function, we first standardize the source code by removing comments from the code snippets. Next, we treat the source code as a natural language sequence to pre-train a Word2vec [69] model. The clean source code is parsed into a code property graph G_i . For the edges in the graph, we convert the directed graph into an undirected graph. For nodes in the graph, the code contained in the node is treated as a token sequence, and its vector representation is initialized using the previously pre-trained Word2vec model. Within a single node, we take the average of all token vectors as the final attribute of that node. Fig. 7 illustrates the code property graph for the first code snippet in our motivational example. This graph intuitively conveys information about syntax structure, control flow, and data dependencies through color-coded elements. Specifically, the blue boxes and lines represent the static syntax structure of the code, where syntax elements from the source code (such as identifiers, operators, and others) are decomposed into nodes based on the AST, resulting in a hierarchical tree-like organization. The orange boxes denote the declaration components within the source code, shedding light on the structural definitions of variables or functions. The red arrows signify the CFG, capturing the potential execution paths of the program during runtime. The green arrows represent the PDG, which reflects the relationships between different parts of the program in terms of data dependencies. Within the CPG, each node is characterized by its type and the information derived from the corresponding code snippet, and we formalize these nodes as the set $\mathcal{V}$. Meanwhile, the AST edges, CFG edges, and PDG edges contribute structural information to the code graph and collectively

form the set $\mathcal{E}$. These nodes and edges are encoded and initialized as feature vectors, which are subsequently utilized as inputs for the model.

4.3 Hierarchical Graph Refinement

In this stage, we introduce an importance-based graph simplification module to hierarchically reduce the size of the graph, where the nodes are filtered based on their importance scores. Considering that the input code property graphs are with various node sizes, we determine whether each code property graph requires undergoing the graph simplification based on a preset threshold. Specifically, if the size of the graph is below the threshold T , the graph will directly proceed to the following graph encoding stage; otherwise, it will go through a number of simplification steps. Such a process is self-adaptive, i.e., it will automatically terminate when the graph size is below the threshold.

4.3.1 The Pooling Operation in One Layer

Pooling is the core operation to reduce the size of the graph. In this subsection, we introduce the pooling operation in ANGEL, named as SAPool, whose workflow is divided into two steps: information aggregation and graph pooling.

The information aggregation step aims to gather structural and semantic information in large code graphs.

The prerequisite of this step is to accurately represent the information of each node. To that end, we employ a two-layer MLP as a projector to map the node information onto a higher-level embedding space, considering the feature extraction capability of MLP. To mitigate the issue of over-fitting during the training process, we also apply a dropout after the MLP. In this way, the model can more accurately capture the characteristics of the source code fragment contained in each node.

Specifically, given a code property graph $G = (\mathbf{X}, \mathbf{A})$ ($\mathbf{X}$ denotes the feature matrix, $\mathbf{A}$ denotes the adjacency matrix), this process can be represented as follows:

$$\mathbf{X} = \text{Dropout}(\mathbf{W}_M(\text{ReLU}(\mathbf{W}_M \mathbf{X}))), \quad (2)$$

where $\mathbf{W}_M$ represents the learnable parameters in the MLP, and $\text{ReLU}(\cdot)$ denotes the activation function. In this process, the self-attribute information of nodes is enhanced.

The next step is information aggregation, aimed at perceiving the structural information within the code graph through the core message-passing operator of GNNs, thereby propagating the features contained between different nodes. Code graphs often exhibit complex structural relationships due to the presence of long-range paths and multiple branching structures. To capture the structural information of the code graph more accurately, we employ APPNP [70] (a well-known information propagation algorithm which denotes *Approximate Personalized Propagation of Neural Predictions*) to aggregate node information. We conducted comparative experiments on APPNP and other GNNs in Section 6. As a variant of GNNs, APPNP is better suited for learning code graphs compared to traditional GNNs (e.g., GGNN, GCN), because APPNP allows for adjustable message-passing ranges, facilitating the aggregation of information from distant graph nodes [71]. In contrast, GNNs like GGNN and GCN focus solely on local neighborhood features, neglecting the long-range dependencies present in code graphs, which limits their effectiveness in tasks such as code vulnerability detection. Moreover, in SAPool, which focuses solely on large code graphs, APPNP does not

require the introduction of additional training parameters, making it more efficient compared to traditional GNNs that introduce new parameters.

Specifically, APPNP executes information aggregation l times in succession based on the graph structure. During each aggregation, it retains the node feature information with a probability of α , and the neighbor feature information aggregated at the current layer is retained with a probability of $(1 - \alpha)$. Finally, these two information resources are added together as the node's feature after this aggregation:

$$\mathbf{X}^{(l)} = (1 - \alpha)\hat{\mathbf{A}}\mathbf{X}^{(l-1)} + \alpha\mathbf{X}^{(0)}, \quad (3)$$

where l represents the number of times APPNP aggregates neighbor information, and $\mathbf{X}^{(l)}$ represents the output features of the APPNP layer. In this way, each node can aggregate neighbor information from 1-th to l -th order, effectively capturing multi-scale contextual information within the graph.

After information aggregation, the model preliminarily extracts the features of the large graph. The core logic of a vulnerability typically involves only a few statements, whereas the entire function may consist of hundreds of lines of code. Other elements unrelated to the vulnerability act as noise, adversely affecting the performance of deep learning models [68]. We employ Top-k pooling [43] to perform hierarchical pooling on large graphs. Compared to traditional global pooling operations (which generate a single one-dimensional vector), Top-k pooling filters redundant information by retaining subgraphs. We add a comparison between Top-k pooling and global pooling in subsequent experiments, which is shown in Section 6. The scoring mechanism of Top-k pooling is similar to the attention mechanism, which aims to help the model gradually pay attention to nodes associated with vulnerabilities, thereby simplifying the size of the code graph. This feature aids the model in filtering redundant information within code graphs, thereby enhancing the accuracy of feature extraction.

Next, we explain Top-k pooling in detail. Specifically, we use a learnable vector $\mathbf{h}$ to map each node feature $\mathbf{X}$ to a one-dimensional space as a score:

$$\mathbf{z} = \mathbf{X} \frac{\mathbf{h}}{\|\mathbf{h}\|}, \quad (4)$$

where $\|\cdot\|$ represents the L2 regularization. The score $\mathbf{z}$ determines the importance of each node. When the score is larger, it means that this node contains more important information. Next, we sort the nodes in descending order based on their scores and manually set a pooling rate K as the proportion of nodes to retain. We select the important node indices according to K :

$$idx = \text{select}(\mathbf{z}, K), K \in (0, 1). \quad (5)$$

Finally, the index is used to construct the adjacency and feature matrices of the simplified subgraph:

$$\mathbf{A} = \mathbf{A}[idx, idx], \quad (6)$$

$$\mathbf{X} = \mathbf{X}[idx, :], \quad (7)$$

$$\mathbf{z} = \mathbf{z}[idx], \quad (8)$$

$$\mathbf{X} = \mathbf{X} \odot (\text{Sig}(\mathbf{z})), \quad (9)$$

where $\text{Sig}(\cdot)$ denotes the sigmoid activation function. It is noted that the computation involves taking the Hadamard product of the projection values $\mathbf{z}$ and the feature matrix $\mathbf{X}$ of the selected top-k nodes. This process is crucial for the network to learn the weight

vector $\mathbf{h}$ during back propagation. In this way, the importance of each node is automatically learned by the network according to the learned vector $\mathbf{h}$. Top-k pooling prompts the model to focus on the nodes that are most influential to the prediction, namely the parts of the code that are most likely to directly trigger vulnerabilities. The key information captured is crucial for vulnerability detection. Thus, the large graph is pooled into a smaller subgraph, containing only the selected important nodes and the edges between them, while many noise nodes in the large graph are filtered out.

4.3.2 Self-Adaptive Pooling Numbers

The above contents introduce the pooling operations in one pooling layer. It should be noted that merely pooling once may not be enough since the graph size might still be larger than the threshold after one SAPool (considering that there might be as many as several thousand nodes in the code graph). To address this challenge, ANGEL adopts a self-adaptive strategy where the number of pooling layers is dynamically determined, i.e., the pooling process is automatically terminated when the number of nodes in the graph is lower than the threshold T (we determine the value of the hyperparameter T in Section 6.3.1). This means that no matter how large the input code graph is, SAPool can simplify it to a code graph with fewer nodes than the threshold T .

Moreover, the pooling ratio of different layers is also dynamically adjusted. For the first SAPool layer, we set K to 0.1, which means only the top ten percentage of the most important nodes are preserved, aiming at quickly filtering the irrelevant information at the beginning. While for the following layers, K is increased by 0.1 each time, and its maximum value is set to 0.5. The behind intuition is that as the code graph simplifies, the proportion of important nodes gradually increases, and the important information might be lost if we maintain a low pooling ratio.

4.4 Context-Aware Graph Encoding

To capture the long-range dependencies in the graph, we designed the global & local feature aggregation module consisting of multiple mixtures of GNN layer and GT layer as the backbone, with the detailed structure shown in Fig. 6. The GNN aims to aggregate local node features through message passing, while the GT aims to aggregate global node features through a multi-head self-attention mechanism.

Local. We employ a general GCN to aggregate information from neighbors. In a single layer GCN, only first-order neighbors can be aggregated. Therefore, the GCN aims to capture local dependencies within the graph structure. The operations in each GCN layer are as follows:

$$\mathbf{X} = \text{ReLU}(\hat{\mathbf{A}}\mathbf{X}\mathbf{W}_G), \quad (10)$$

where $\mathbf{W}_G$ denotes the learnable parameters in the GCN. Note that in the previous hierarchical graph refinement stage, the large graph may have been pooled into a non-connected graph. Solely using a GCN cannot extend information propagation to non-connected graphs and is limited in terms of long-distance features. To address this, we introduce GT to learn graph representations from a global perspective.

Global. We designed a GT based on multi-head self-attention mechanism. The fully connected learning mechanism inside has the following two advantages: (1) it captures long-distance dependencies between nodes; (2) it learns potential associations between

subgraphs within non-connected graphs. The GT is divided into two steps: multi-head self-attention (MHA) and feed-forward blocks (FFN). First, in a single attention head, the attention score matrix $\mathbf{S}$ of each node is learned by all nodes, and the score matrix is utilized to update the node features:

$$\mathbf{S} = \text{softmax} \left(\frac{(\mathbf{X}\mathbf{W}_Q)(\mathbf{X}\mathbf{W}_K)^T}{\sqrt{d}} \right), \quad (11)$$

$$\mathbf{X}_i = \mathbf{S}\mathbf{X}\mathbf{W}_V, \quad (12)$$

where $\mathbf{W}_Q$, $\mathbf{W}_K$, and $\mathbf{W}_V$ are the learnable parameter matrices in the MHA. The outputs $\mathbf{X}$ from different heads are concatenated to obtain the final output of MHA:

$$\mathbf{X} = \text{Concat}(\mathbf{X}_1, \mathbf{X}_2, \dots, \mathbf{X}_i). \quad (13)$$

Through this method, the global attention in the graph is captured, and the information of each node is updated by the features from all the other nodes in the graph. Next, the output from MHA is fed into the FFN, and residual connections are added respectively in MHA and FFN:

$$\mathbf{X} = \text{MHA}(\mathbf{X}) + \mathbf{X}, \quad (14)$$

$$\mathbf{X} = \text{FFN}(\mathbf{X}) + \mathbf{X}. \quad (15)$$

The FFN consists of a two-layer MLP that expands and then compresses dimensions:

$$\mathbf{X} = \mathbf{W}_F(\text{LN}(\mathbf{X})), \quad (16)$$

where $\text{LN}(\cdot)$ represents layer normalization [72], $\mathbf{W}_F$ represents the learnable parameters in the FFN.

We add residual connections twice within the GNN-GT framework to enhance the propagation of feature information. Thanks to the complementary feature propagation of GNN and GT layers, both global and local information are adequately exploited to facilitate the extraction of vulnerability information in code graphs.

Finally, we compress the node representation obtained by backbone network encoding into a one-dimensional vector for classification based on global average:

$$\mathbf{o} = \text{AvgPool}(\mathbf{X}). \quad (17)$$

4.5 Classification

In the final stage of the paradigm, we feed the previously obtained one-dimensional vector $\mathbf{x}$ into a two-layer MLP, and then directly output the probabilities of two categories (vulnerable and non-vulnerable). We use binary cross-entropy loss to estimate the loss between the predicted labels and the actual labels:

$$\mathcal{L} = -\frac{1}{M} \sum_{i=1}^M [\mathbf{y}_i \log(\mathbf{p}_i) + (1 - \mathbf{y}_i) \log(1 - \mathbf{p}_i)], \quad (18)$$

where $\mathcal{L}$ is the loss, M is the number of graphs, $\mathbf{y}_i$ is the actual label of the i -th graph, and $\mathbf{p}_i$ is the predicted probability of the i -th graph as a vulnerable code snippet. During the training process of ANGEL, we perform forward propagation to calculate the loss and update the learnable parameters in the model through backward propagation.

TABLE 1: Statistics of datasets.

Dataset	Vulnerable	Non-vulnerable	Total	Language
Devign	10067	12294	22361	C/C++
Reveal	1664	16511	18175	C/C++
Big-Vul	10900	177736	188636	C/C++

5 EXPERIMENTAL SETUP

5.1 Research Questions

To verify the effectiveness of ANGEL, we aim to answer the following research questions:

RQ1: Effectiveness. Can ANGEL surpass current state-of-the-art learning-based vulnerability detection methods?

RQ2: Rationale. To what extent do different design choices affect the overall performance of ANGEL?

RQ3: Sensitivity. To what extent do different hyper-parameters affect the overall performance of ANGEL?

5.2 Experimental Setup

5.2.1 Datasets

We evaluate the effectiveness of the method using three widely used public vulnerability datasets, namely Devign [26], Reveal [30], and Big-Vul [31]. The distribution information of the three datasets is shown in Table 1. The Devign dataset consists of two manually labeled real-world projects, FFMpeg and Qemu. It contains a total of about 22k samples, with the proportion of vulnerable samples being 45.02%. The Reveal dataset comes from two open-source projects, Debian and Chromium, and contains a total of about 18k samples, with the vulnerability proportion being 10.04%. Big-Vul is collected from a large number of GitHub projects, including 91 different types of vulnerabilities, with a vulnerability proportion of 5.77%.

5.2.2 Baseline Methods

In our experiments, we compared ANGEL with fourteen state-of-the-art vulnerability detection methods, including three static analysis approaches, three token-based techniques, and eight graph-based methods. Detailed descriptions of the baseline methods used for comparison are provided below.

(1) **Flawfinder** [73]: Flawfinder is a static analysis tool that integrates rules to identify C/C++ source code functions that are susceptible to vulnerabilities.

(2) **RATS** [74]: RATS is an open-source static code analysis tool that identifies potential vulnerabilities by matching against predefined patterns, such as dangerous function calls and unsafe API usage, and subsequently generates a report.

(3) **Cppcheck** [75]: Cppcheck is an open-source static code analysis tool specifically designed for C and C++, with a deep understanding of the nuances and common pitfalls associated with both languages.

(4) **VulDeePecker** [17]: VulDeePecker utilizes a bidirectional LSTM to learn the natural language sequence representation of code for vulnerability detection.

(5) **SySeVR** [18]: SySeVR utilizes a bidirectional RNN to learn the program representation of vulnerability-related syntactic and semantic information.

(6) **PILOT** [18]: PILOT leverages Positive and Unlabeled (PU) learning to mitigate the impact of noise within data for vulnerability detection.

(7) **Devign** [26]: This method combines multiple graph structures as input and uses GGNN and CNN to extract features for vulnerability detection.

(8) **Reveal** [30]: Reveal consists of a feature extraction and training phase. It first parses the code into a Code Property Graph, then builds the model using GRU and GGNN.

(9) **IVDetect** [38]: IVDetect takes the PDG as input and uses GCN for vulnerability detection.

(10) **DeepWuKong** [41]: DeepWuKong extracts semantic features of the code through program slicing methods and uses GNN to learn structured information for vulnerability detection.

(11) **MAGNET** [29]: MAGNET views the code structure graph as a heterogeneous graph and uses a meta-path-based attention graph neural network to extract features.

(12) **Coca** [23]: Coca integrates self-supervised with supervised contrastive learning to acquire robust feature representations, thereby enhancing the effectiveness of vulnerability detection. Notably, the Coca integrates multiple models. In this paper, we employ its optimal configuration (Coca+Reveal) as the baseline.

(13) **MGVD** [76]: MGVD employs raw text, abstract graphs, and raw graphs as inputs to learn semantic and structural information in code from multiple perspectives.

(14) **AMPLE** [37]: AMPLE is based on manually defined rules to reduce the size of the code structure graph and uses multi-head attention mechanisms and CNN to learn general graph representations.

Please note that the baselines are all open-sourced, and we directly reuse their implementations in this study to avoid potential threats in the reproduction process.

5.2.3 Evaluation Metrics

To comprehensively assess the performance of our proposed method, we employ the following standard evaluation metrics: Accuracy, Precision, Recall, and the F1 score. Each metric offers insight into different aspects of the model's prediction capabilities.

Precision: $\text{Precision} = \frac{TP}{TP + FP}$. Precision measures the ratio of correctly predicted positive observations to the total predicted positive observations. High precision relates to a low rate of false positives.

Recall: $\text{Recall} = \frac{TP}{TP + FN}$. Recall measures the ratio of correctly predicted positive observations to all observations in actual class. Recall emphasizes the model's ability to find all relevant instances in the dataset.

F1 score: $F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$. F1 score is the weighted average of precision and recall. Therefore, this score takes both false positives and false negatives into account. It is especially useful in the case of uneven class distribution.

Accuracy: $\text{Accuracy} = \frac{TP + TN}{TP + TN + FN + FP}$. Accuracy is the most intuitive performance measure and it is simply the ratio of correctly predicted instances to the total instances in the dataset.

5.3 ANGEL Implementation

We implement ANGEL using the Pytorch and PyTorch Geometric libraries in Python 3.10. The model is trained on a single NVIDIA GeForce RTX 4090 GPU. For the initial dimension of Word2vec, we follow the setting of the previous method [37] and set it to 100. In terms of parameter settings for the model, we mainly focused on the threshold T (10, 20, 30, 40, 50, 60, 70, 80, 90, 100), aggregation times l (2, 4, 6, 8, 10), retention probability α (0.1, 0.2, 0.4, 0.6, 0.8), GNN-GT layers (2, 3, 4, 5, 6, 7, 8), head

numbers (1, 2, 4, 8, 16). Finally, the threshold T , aggregation times l , retention probability α , GNN-GT layers and head numbers are empirically determined to 40, 8, 0.2, 5, 4, respectively. Moreover, the hidden dimension of the model is set to 64. During the model training phase, the batch size is set to 1024, with a maximum iteration count of 3000. AdamW [77] is used as the optimizer, with a learning rate of 1e-3 and weight decay of 1e-5. We replicate all baseline models based on publicly released source codes and use the same hyper-parameter settings as described in their original papers. Finally, to ensure a fair comparison, all methods are subjected to the same partitioning standard. The datasets are randomly divided into training, validation, and testing sets in a 7:1:2 ratio.

This experimental setup aligns with that of the Reveal study [30], which aims to enhance the testing dataset size to evaluate the generalizability of the approaches. Please note that the original AMPLE study splits the dataset based on a 8:1:1 ratio [37]. Consequently, there exists a disparity between the outcomes acquired in our study and those initially reported. Specifically, the initially reported performance of AMPLE slightly surpasses the results obtained in our experiments (as will be demonstrated in Section 6), possibly due to the availability of more data for comprehensive training.

6 EXPERIMENTAL RESULT

6.1 Comparison with State-of-the-Arts (RQ1)

To address the first research question, we compared ANGEL with eight state-of-the-art baseline methods across three datasets. Subsequently, we visualized the latent variables in the context-aware graph encoding stage to further validate the effectiveness of ANGEL.

6.1.1 Comparison of Results

The comparison analysis in Table 2 demonstrates the overall performance results of ANGEL across four evaluation metrics. Such comparisons have led us to some key insights. First, ANGEL consistently surpasses the proposed baseline methods in ACC and F1 scores across all three datasets, reflecting its superior effectiveness. Compared to static analysis methods, ANGEL demonstrates a more balanced advantage in terms of performance. Although the best-performing static analysis tool, Cppcheck, achieves a higher recall rate on the Reveal and Big-Vul datasets, its elevated false positive rate results in comparatively lower precision and F1 scores. A high false positive rate not only increases the workload for developers but may also lead to the neglect of critical vulnerabilities, thereby diminishing the efficiency of vulnerability remediation in practical applications. In contrast, ANGEL exhibits significantly better precision than Cppcheck, effectively reducing false positives while maintaining a high recall rate and F1 score. This makes ANGEL stand out in overall performance, particularly by providing more stable and reliable vulnerability detection results in real-world scenarios. Compared to token-based methods, ANGEL shows significant improvements across four evaluation metrics on three datasets. Specifically, ANGEL achieves an absolute improvement of 14.71% in ACC and 21.07% in F1 over the token-based approach VulDeePecker on the Big-Vul dataset. PILOT, leveraging the pre-trained natural language model CodeBERT and PU learning, has established itself as the best baseline among token-based approaches. However, compared to PILOT, ANGEL achieves an average absolute improvement of

TABLE 2: Comparison results between ANGEL and the baselines. The **boldface** values indicate the best results (%).

Type	Dataset	Devign				Reveal				Big-Vul			
	Method	Accuracy	Precision	Recall	F1 score	Accuracy	Precision	Recall	F1 score	Accuracy	Precision	Recall	F1 score
Static Analysis	Flawfinder	54.00	47.31	14.67	22.39	79.89	14.79	21.01	17.36	88.72	12.45	16.53	14.20
	RATS	52.93	47.01	21.47	29.48	77.36	12.66	21.23	15.86	86.72	11.23	19.58	14.27
	Cppcheck	56.54	54.52	30.98	39.51	74.75	18.56	44.64	26.22	69.67	9.96	54.37	16.83
Token-based	VulDeePecker	46.46	44.78	42.73	43.73	69.49	8.87	23.13	12.82	74.76	6.56	25.29	10.42
	SySeVR	49.44	43.56	39.11	41.21	77.47	13.50	23.50	17.15	77.84	9.32	32.95	14.53
	PILOT	52.47	51.38	53.74	52.53	81.37	21.99	31.83	26.40	85.41	21.83	19.35	20.51
Graph-based	Devign	48.13	45.11	49.10	47.02	76.71	11.30	19.00	14.17	79.73	9.10	24.14	13.22
	DeepWuKong	50.19	46.40	49.92	48.09	81.25	18.67	25.42	21.53	77.74	9.81	30.31	14.83
	Reveal	49.85	46.78	50.70	48.66	81.30	18.18	24.23	20.77	82.81	13.38	30.84	18.66
	IVDetect	50.66	47.36	47.25	47.30	79.93	16.88	24.82	20.10	79.37	11.15	31.95	16.53
	MAGNET	51.43	48.55	60.77	53.98	82.17	23.82	34.68	28.24	81.22	12.96	33.90	18.75
	Coca	49.67	46.93	56.17	51.13	80.44	18.46	27.32	22.03	82.31	15.97	26.78	20.01
	MGVD	54.32	51.25	52.26	51.75	81.24	25.77	30.08	27.76	83.02	14.41	34.53	20.34
Graph-based	AMPLE	53.12	50.00	55.92	52.80	82.79	26.99	41.09	32.58	83.60	15.77	36.06	21.95
	ANGEL	58.87	55.08	66.41	60.22	87.45	37.77	37.05	37.41	89.47	26.96	37.85	31.49

9.89% across three datasets. This is because token-based methods treat code as simple natural language sequences, overlooking structural information. In contrast, graph-based methods can better capture structural information in code, which is advantageous for vulnerability detection. Despite this, ANGEL still outperforms the existing graph-based approaches. Compared with the best-performing graph-based method AMPLE on the Big-Vul dataset, ANGEL achieves an absolute increase of 5.87% in ACC and 9.54% in F1 score. Although AMPLE reduces the number of nodes to some extent by using artificially defined rules, the proportion of simplification based on these rules might be less than 20% (according to our manual investigations). As a comparison, our ANGEL can reduce the size of the graph by more than 70% on average (as will be discussed in Section 7). In code graphs, AMPLE still faces the problem of the large amount of noise and limited understanding ability towards long-range dependencies, resulting in suboptimal outcomes. When compared with the most trivial graph-based baseline method Devign in terms of F1 on three datasets, ANGEL achieves an absolute increase of 13.2%, 23.24%, and 18.27%. This is because Devign merely transforms the code into a graph structure and then trains the GNN model merely based on the original code graphs, thus facing greater challenges when the code graphs are complex. Compared to recent baselines such as Coca and MGVD, ANGEL demonstrates comprehensive superiority. For instance, on the Big-Vul dataset, ANGEL achieves absolute improvements of 11.48% and 11.15% over these methods, respectively. Overall, when considering all the 12 settings (4 performance metrics $\times$ 3 datasets), ANGEL achieves the best performances in 10 cases.

6.1.2 Visualization of Results

To directly observe the effectiveness of ANGEL, we used the T-SNE algorithm [78] to visualize the graph-level embeddings, helping us understand how different categories of data are distinguished in the latent space. We selected four baseline methods for embedding visualization on the Devign dataset. The experimental results are shown in Fig. 8, where red dots represent vulnerable samples and blue dots represent non-vulnerable samples. When using the T-SNE algorithm for embedding visualization, it is usually expected that data of different categories form relatively independent cluster centers in the low-dimensional space. Based on Fig. 8, we make the following observations: (1) The Devign method entangles the two categories of data. This indicates that Devign fails to adequately capture the features distinguishing the

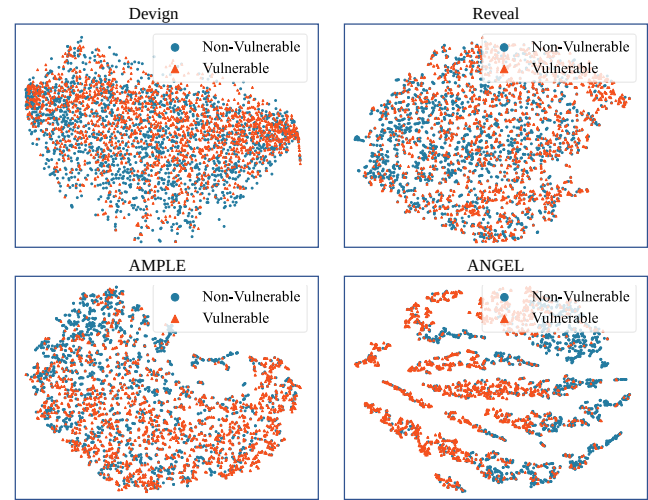


Fig. 8: The T-SNE algorithm [78] visualization illustrates the distribution between vulnerable and non-vulnerable samples in the code representations of different methods.

different categories, making it difficult to separate them in the latent space; (2) The performance of Reveal is similar to that of Devign, as it also confuses the two categories of data in the embedding space; (3) AMPLE performs better than the previous two methods, with more non-vulnerable samples clustering in the top-left corner and more vulnerable samples clustering in the bottom-right corner. This suggests that AMPLE has stronger feature extraction capabilities for both types of samples. However, overall, AMPLE does not distinctly separate the two categories of samples; (4) ANGEL shows obvious separability under visual observation, where the vulnerable samples are mainly concentrated on the left, and the non-vulnerable samples are mainly concentrated on the right. Relative to the three baseline methods, the embeddings generated by ANGEL are more distinguishable in the latent space. Such results demonstrate the effectiveness of ANGEL in distinguishing vulnerable and non-vulnerable codes.

Answer to RQ1: ANGEL exhibits improved effectiveness, performing better than all baseline methods in terms of ACC and F1 scores. Compared to the best-performing baseline AMPLE, ANGEL achieves a 14.0%-43.4% increase in terms of the F1 scores across three datasets. Visualization further indicates that ANGEL captures key information in the code more effectively than baseline methods.

6.2 Ablation Analysis (RQ2)

6.2.1 The impact of different modules

To answer the second research question, we conducted ablation studies on three datasets to explore the impact of each component in ANGEL on the results. We compare three different variants of ANGEL: (1) without hierarchical graph refinement (denoted as “w/o HGR”); (2) without local aggregation in GNN (denoted as “w/o GNN”); (3) without global aggregation in GT (denoted as “w/o GT”). The experimental results are shown in Table 3, with the performance of all the variants being lower than ANGEL, indicating that every component contributes to the overall performance of ANGEL. Specifically, several key observations were summarized: (1) For the variant “w/o HGR”, the accuracy, precision, recall, and F1 scores decline by an average of 4.56%, 5.67%, 10.48%, and 7.77% across the three datasets, respectively, which are the most significant impacts on the overall performance. This suggests that hierarchical graph refinement plays a crucial role in vulnerability detection, helping the models concentrate on critical information; (2) For the variant “w/o GNN”, there was a decrease of 0.98%, 1.63%, 2.80%, and 2.25% respectively in the four metrics, indicating that GNNs are effective in aggregating neighbor information during the feature extraction process; (3) Compared to the variant “w/o GT”, ANGEL showed improvements of 2.40%, 2.11%, 5.51%, and 3.55% respectively in the four metrics. This demonstrates that GT can overcome the limitations of GNNs, extracting potential global dependencies in the code structures.

6.2.2 The Impact of APPNP in SAPool

In SAPool, we introduce an algorithm called APPNP, designed to perform information aggregation in code graphs. We select three classic GNNs (i.e., GGNN [27], GCN [79], and GAT [80]) as variants to replace APPNP in order to observe the changes in detection performance. Table 4 presents the F1 scores of different variants across three datasets, and based on these, we observe the following: (1) GCN is simple, efficient, and has low computational complexity, but it yields poor results when applied to SAPool, with F1 scores of 53.61%, 33.56%, and 29.06% on the three datasets, respectively; (2) GGNN and GAT dynamically propagate and aggregate information from neighboring nodes through gated recurrent units and attention mechanisms, respectively, achieving superior results compared to GCN; (3) When APPNP is used, the model achieves the best results, with F1 scores of 60.22%, 37.41%, and 31.49% on the three datasets, respectively. This is because GCN, GGNN, and GAT focus solely on learning from first-order neighboring nodes, failing to capture the structural relationships between higher-order neighbors. This suggests that relying exclusively on these traditional GNNs makes it challenging to extract long-range dependency features, i.e., control flow and data flow, from code graphs. In contrast, APPNP possesses the

ability to aggregate long-distance features, enabling the model to better learn the feature within the code graph.

6.2.3 The Impact of Top-k in SAPool

To validate the effectiveness of the Top-k pooling strategy, we select three different global pooling methods as baselines: global average pooling (denoted as “Avg”), global sum pooling (denoted as “Sum”), and global max pooling (denoted as “Max”). “Avg” and “Sum” compute a one-dimensional vector by averaging or summing all node features, respectively. “Max” retains the maximum feature value among all nodes as a new one-dimensional vector. Table 5 illustrates the F1 scores of different pooling strategies across three datasets. It is evident that the detection performance of the model under the three global pooling baselines is significantly lower than that of Top-k pooling. Specifically, both “Avg” and “Sum” force all semantic and structural information of nodes in the graph to be compressed into a one-dimensional vector, causing confusion in node information and thus resulting in poor detection performance. “Max” considers only the maximum value of each feature dimension, resulting in a significant loss of node information in the pooled vector, which leads to the worst performance among the various baselines. Compared to previous global pooling methods, Top-k pooling is more suitable for learning large code graphs. During each pooling operation, it compresses the code graph into a smaller subgraph, preserving pivotal node information relevant to the categories. Through a hierarchical pooling process, large code graphs are simplified into smaller subgraphs, making it easier for subsequent graph encoding models to extract the features of the code graph.

Answer to RQ2: Each component of ANGEL effectively brings positive gains to the model, highlighting the rationale of the proposed components. Among them, hierarchical graph refinement contributes the most to the overall performance.

6.3 Hyper-Parameter Sensitivity Analysis (RQ3)

In this section, we explore the impact of hyper-parameters on the performance of ANGEL, including settings of the pooling ratio of SAPool, hyper-parameter configurations of APPNP, and the number of layers and attention heads in GNN-GT.

6.3.1 The Impact of Threshold in the Importance-based Graph Simplification Module

We explore the impact of different threshold T in the importance-based graph simplification module on ANGEL across three datasets. According to Table 6, we can observe that: (1) When the threshold is set between 30 and 60, ANGEL achieves better performance; (2) When the threshold is set too small, F1 score of ANGEL will decrease. This is because retaining too few nodes may not be able to involve enough vulnerability-related information, leading to the loss of important information in the graph; (3) When the threshold is set too high, the code graph is not sufficiently simplified, making it difficult for the model to learn the vulnerability pattern. Overall, based on observations from the three datasets, setting the threshold T to 40 allows ANGEL to achieve the optimal results.

TABLE 3: Results of ablation study.

Module	Devign				Reveal				Big-Vul			
	Accuracy	Precision	Recall	F1 score	Accuracy	Precision	Recall	F1 score	Accuracy	Precision	Recall	F1 score
“W/O HGR”	55.82	52.91	52.38	52.64	82.87	30.63	31.35	30.99	83.42	19.25	26.15	22.18
“W/O GNN”	57.96	54.48	62.71	58.31	86.36	36.64	34.20	35.38	88.54	23.81	36.00	28.67
“W/O GT”	56.75	53.32	62.05	57.35	85.21	35.66	32.78	34.16	86.63	24.51	29.94	26.96
ANGEL	58.87	55.08	66.41	60.22	87.45	37.77	37.05	37.41	89.47	26.96	37.85	31.49

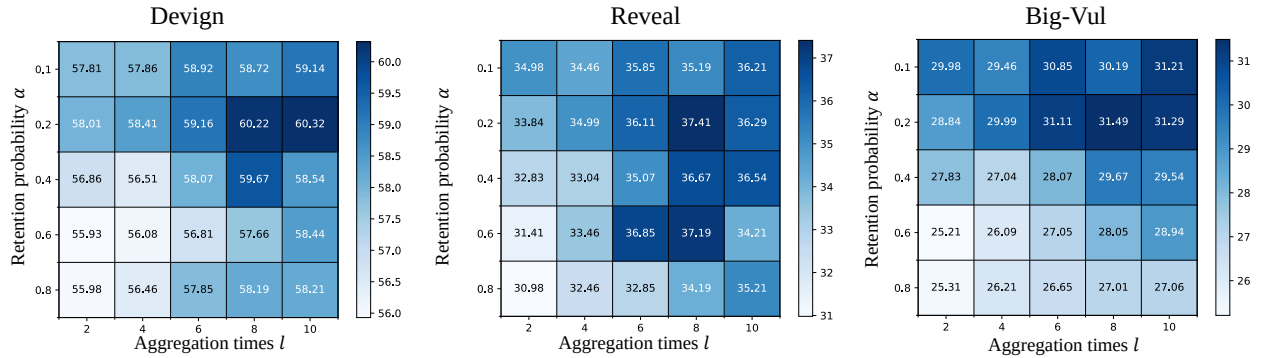


Fig. 9: The F1 score of different parameter settings in the APPNP layer.

TABLE 4: The impact of different GNNs on SAPool.

Method	Devign	Reveal	Big-Vul
GCN	53.61	33.56	29.06
GGNN	57.49	36.27	29.70
GAT	55.24	34.37	30.46
APPNP	60.22	37.41	31.49

TABLE 7: The F1 score of different pooling rates K in Top-k pooling.

Pooling rate K	Devign	Reveal	Big-Vul
Fixed to 0.1	56.20	34.21	28.80
Fixed to 0.5	59.94	37.10	31.68
Increment from 0.1 to 0.5	60.22	37.41	31.49

TABLE 5: The impact of different pooling strategies on SAPool.

Method	Devign	Reveal	Big-Vul
Avg	47.57	28.57	24.53
Sum	48.68	27.03	22.92
Max	42.64	21.05	20.60
Top-k	60.22	37.41	31.49

6.3.2 The Impact of Preset Parameters in APPNP

Eq. (3) introduces two hyper-parameters, l and α , to influence the distance of message propagation and the importance of the node information. Fig. 9 shows the effects of hyper-parameters l and α on the network. Specifically, we can observe the following

TABLE 6: The F1 scores under different thresholds T in the importance-based graph simplification module.

Threshold T	Devign	Reveal	Big-Vul
10	53.39	30.92	29.66
20	57.43	33.75	29.99
30	59.05	37.06	30.38
40	60.22	37.41	31.49
50	59.22	36.56	30.15
60	57.15	35.82	29.28
70	55.40	35.52	28.96
80	53.83	33.95	26.28
90	52.60	30.70	25.54
100	51.55	29.14	24.13

phenomenons: (1) The performance of ANGEL generally improves as l increases. On the Devign dataset, optimal performance is achieved when l reaches 10. On the Reveal and Big-Vul datasets, growth gradually plateaus when l reaches 8; (2) The performance of ANGEL enhances as α decreases, reaching its peak when α reaches 0.2 across all three datasets. Overall, to unify the hyper-parameters of ANGEL as much as possible, we set the l and α to 8 and 0.2, respectively, enabling more effective capture of structural information within the graph.

6.3.3 The Impact of pooling rate in SAPool

We investigate the impact of different pooling rates in SAPool on ANGEL across three datasets. We set three different pooling rates: fixed at 0.1, fixed at 0.5, and incrementally increasing from 0.1 to 0.5. As observed in Table 7, the following conclusions can be drawn: (1) When the pooling rate in SAPool is consistently set to 0.1, ANGEL performs the worst. This is due to the fact that maintaining such a small pooling rate continuously compresses the code graph into overly small subgraphs, leading to significant information loss; (2) When the pooling rate is fixed at 0.5, a significant performance improvement is observed compared to 0.1, achieving performance comparable to that of incremental pooling rates. However, this fixed pooling rate of 0.5 leads to an increase in the number of adaptive pooling operations, resulting in an increase in model parameters and a reduction in efficiency. This is because ANGEL needs to ensure that the number of nodes in the simplified code graph remains below the threshold T ; (3) As

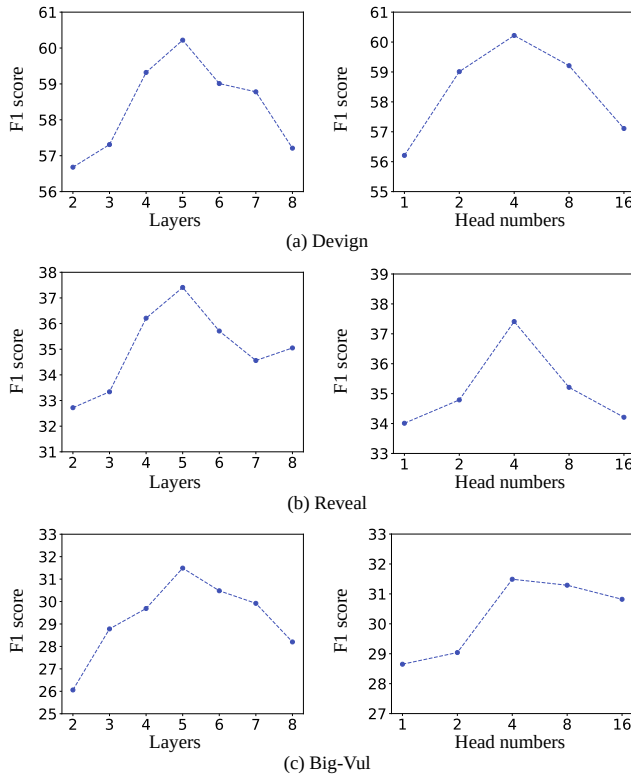


Fig. 10: Parameter analysis of GNN-GT layers and head numbers of GT.

the pooling rate gradually increases from 0.1 to 0.5, the model achieves optimal performance on both datasets (i.e., Reveal and Big-Vul) while having fewer pooling layers than when K is fixed at 0.5. Therefore, we set the pooling rate to an increasing sequence from 0.1 to 0.5.

6.3.4 The Number of Layers and The Number of Attention Heads in GNN-GT

We analyzed the impact of different numbers of layers and attention heads in GNN-GT on the performance of ANGEL. As illustrated in Figure 10, we observe that the hyperparameter trends across the three datasets exhibit a pronounced hump-shaped pattern. One possible explanation for this is that when the layer count and number of attention heads in GNN-GT are too low, the feature extraction capability of ANGEL is constrained, resulting in subpar overall performance. On the other hand, excessively increasing the layer count and attention heads may introduce risks of overfitting. By analyzing the trends across all three datasets, we find that ANGEL achieves best performance when the number of GNN-GT layers is set to 5 and the number of attention heads is configured to 4.

Answer to RQ3: The setting of hyper-parameters affects the performance of ANGEL across multiple datasets. To simplify the parameter configuration, we set the same hyper-parameters on different datasets while achieving relatively good performance.

TABLE 8: The accuracy of ANGEL and variants in different node number ranges on the Big-Vul [31] dataset.

Method	(0, 25]	(25, 50]	(50, 100]
“w/o HGR”	95.51	93.15	84.49
ANGEL	96.89	95.23	91.66

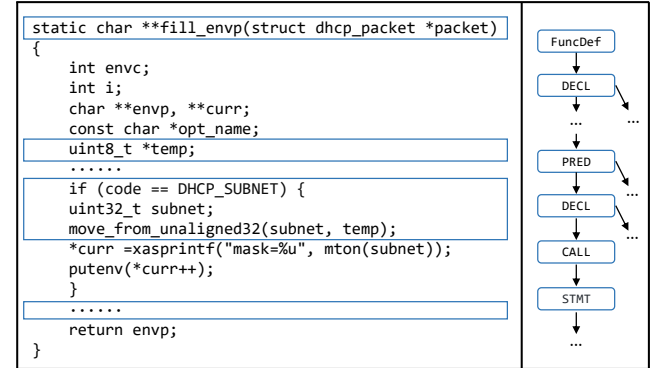


Fig. 11: An example of graph simplification. The blue boxes represent the retained statements after simplification.

7 DISCUSSION

7.1 Why Dose ANGEL Work?

7.1.1 Advantages of ANGEL

We identify that ANGEL has two advantages that can explain its effectiveness in source code vulnerability detection.

(1) The ability for adaptive simplification of code graphs.

The proposed hierarchical graph refinement stage simplifies graphs with a large number of nodes into subgraphs with critical nodes, helping ANGEL more effectively capture key information in large graphs. Specifically, we calculate the average number of nodes in the input and output of the hierarchical graph refinement stage. Taking the Big-Vul [31] dataset as an example, the number of nodes in the simplified graph decreases by an average of 75.9% compared to the original input, from 104 to 25. Additionally, we explored the impact of the hierarchical graph refinement stage on samples with different numbers of nodes in the Big-Vul dataset. Similarly, the variant model configuration was “w/o HGR”, and the experimental results are shown in Table 8. We found that when the number of nodes is (0, 25], the gap between ANGEL and “w/o HGR” is only 1.38%. As the node count interval increases, the gap between ANGEL and “w/o HGR” also gradually increases. In the range where the number of nodes exceeds 50, the performance gap increases from 1.38% to 7.17%. This indicates that hierarchical graph refinement can effectively mine critical information in large graphs, thereby making the model generalize well to complex code graphs. An example is shown in Fig. 11 where we demonstrate the simplified code graph of the second code snippet from our motivating examples in Section 3. The initial graph comprises over 400 nodes, whereas in the simplified version, it condenses to just 21 nodes. Nevertheless, as depicted in the graph, the majority of retained nodes are vulnerability-relevant, such as those associated with the declaration and invocation of the `temp` variable, which leads to the vulnerability.

(2) The capability to perceive local and global contexts information. We have designed a module that aggregates both global and local features, aiming at learning informative content within the graph. It utilizes GNN to aggregate local information

```

1.  zsethalfone5(i_ctx_t *i_ctx_p)
2.  {
3.      os_ptr op = osp;
4.      uint count;
5.      .....
18. -  int dict_enum = dict_first(op);
   +  int dict_enum;
   .....
28. -  int space_index = r_space_index(op - 1);
   +  int space_index;
   +  if (ref_stack_count(&o_stack) < 2)
   +      return_error(gs_error_stackunderflow);
   +  check_type(*op, t_dictionary);
   +  check_type(*(op - 1), t_dictionary);
   +  dict_enum = dict_first(op);
   +  space_index = r_space_index(op - 1);
   .....
283. }

```

Fig. 12: A source code of CWE-704 example. The red-colored code is vulnerable code, and the green-colored code is the repaired code.

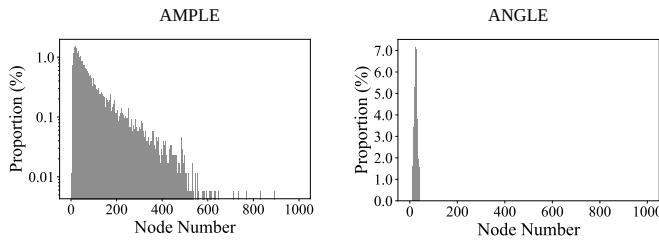


Fig. 13: The statistical distribution of the sizes of simplified code graphs for AMPLE and ANGEL on the Reveal dataset.

and employs GT to aggregate global information. The GT assists the GNN in capturing long-distance dependencies and avoiding the over-smoothing problem. GNN helps the GT to enhance local information, preventing important neighbor features from being overwhelmed by global features. As demonstrated in Table 3, when GNN and GT are used independently, the detection performance of model decreases in both cases. Such results suggest that there is a complementarity between GNN and GT. Therefore, the GNN-GT layer that we designed by combining GNN and GT effectively captures the structural relationships within the graph, thereby enhancing the performance of vulnerability detection.

7.1.2 The Impact of Different Code Graph Sizes on Detection Results

In Fig. 2, we present the results of ANGEL and the state-of-the-art method AMPLE on code graphs of varying sizes. Next, we discuss the reasons behind the performance differences between AMPLE and ANGEL on large graphs. AMPLE removes nodes with redundant semantics during the data pre-processing phase based on manually predefined rules. However, due to the limitations of

these rules, this approach can only simplify a small number of redundant nodes. As shown in Fig. 13, we illustrate the size of the simplified code property graphs of AMPLE and ANGEL on the Reveal dataset. We can visually observe that AMPLE still contains a significant number of samples with more than 100 nodes in the simplified code graphs, while in contrast, ANGEL reduces all the code graphs to fewer than 40 nodes. Our statistics reveal that the average node count for AMPLE in the simplified code graphs is 91, whereas ANGEL's average node count is only 26. Overall, ANGEL demonstrates a significantly superior ability to compress graph size compared to AMPLE. This difference leads to overall performance of AMPLE being inferior to ANGEL across multiple datasets and evaluation metrics. Furthermore, as shown in Fig. 2, we observe a noticeable performance gap between AMPLE and ANGEL when the node count exceeds 100. We conducted further analysis on the compression of code graph sizes by AMPLE and ANGEL when the node count exceeds 100. As illustrated in Table 9, AMPLE, which is based on a rule-based graph simplification method, faces limitations when handling large code graphs. Specifically, as the original code graph size increases, AMPLE's simplification performance gradually diminishes. When the node count exceeds 300, the average node count in AMPLE's simplified graphs remains as high as 254. In contrast, ANGEL employs a hierarchical graph refinement strategy, which can effectively adapt to code graphs of varying sizes, resulting in superior detection performance. Thus, the performance gap between AMPLE and ANGEL grows as the node count increases.

7.2 Performance comparison with commercial large language models

In recent years, large language models (LLMs) have made remarkable strides in natural language processing and code analysis. Commercial models like ChatGPT [81] have demonstrated exceptional generalization and contextual understanding, thanks to their immense parameter sizes and extensive pre-training data. To evaluate the performance differences between the proposed ANGEL and existing LLMs, we conducted a thorough comparative analysis using DeepSeek-V3 [82] and GPT-4o [83] as baselines, with the experimental results presented in Table 10. For prompt design, we adopt a zero-shot setting following the configurations of prior work [84], [85], requiring the LLM to directly determine whether the given code snippet contains vulnerabilities. As shown in the table, ANGEL consistently outperforms GPT-4o and DeepSeek-V3 across all datasets, particularly in terms of accuracy and F1 score. Specifically, on the Devign dataset, ANGEL achieves an F1 score of 60.22%, representing improvements of 18.96% and 8.64% over DeepSeek-V3 and GPT-4o, respectively. Similarly, on the Reveal and Big-Vul datasets, ANGEL attains F1 scores of 37.41% and 31.49%, substantially surpassing its counterparts. Notably, although GPT-4o exhibits the highest recall across the three datasets, it also manifests a significantly higher false positive rate. Although LLMs excel in understanding code, they are not particularly adept at vulnerability detection tasks. Furthermore, different LLMs may exhibit overly conservative or overly aggressive behavior when predicting the presence of vulnerabilities, resulting in relatively low F1 scores. This conclusion aligns with a previous study. Specifically, Gao et al [86] found that GPT-4 achieves an F1 score of only around 20% on real-world vulnerability detection tasks. Overall, ANGEL's performance remains superior to that of LLMs.

TABLE 9: Comparison of the average number of nodes after simplification between ANGEL and AMPLE in different node number ranges.

Interval	Original	AMPLE	ANGEL
[100, 200)	143	76	29
[200, 300)	244	125	25
[300, 1000)	511	254	30

TABLE 10: Comparison results between ANGEL and the LLM.

Method \ Dataset	Devign				Reveal				Big-Vul			
	Accuracy	Precision	Recall	F1 score	Accuracy	Precision	Recall	F1 score	Accuracy	Precision	Recall	F1 score
DeepSeek-V3	53.94	49.75	51.52	50.62	72.99	10.94	28.61	15.83	76.20	10.28	41.60	16.48
GPT-4o	50.85	47.42	66.69	55.43	54.93	11.80	53.83	19.36	62.32	8.55	58.54	14.93
ANGEL	58.87	55.08	66.41	60.22	87.45	37.77	37.05	37.41	89.47	26.96	37.85	31.49

TABLE 11: Comparison of resource consumption between ANGEL and the baselines.

Method	Parameters (M)	Time Consumption (s)	F1 Score
Devign	0.76	11.7	13.22
Reveal	1.07	13.8	18.66
Coca	1.11	18.4	20.01
MGVD	1.97	80.6	20.34
AMPLE	1.73	110.5	21.95
ANGEL	0.30	24.2	31.49

7.3 Limitation

Despite ANGEL achieving superior performance over baseline methods in vulnerability detection tasks, its F1 scores on the Reveal [30] and Big-Vul [31] datasets are only 37.4% and 31.5%, respectively. The reason for this phenomenon is the extreme imbalance between vulnerable and non-vulnerable samples in the datasets, with different types of vulnerabilities within the vulnerable samples following a long-tail distribution. Specifically, taking the Big-Vul dataset as an example, CWE-119 accounts for 19.94% of vulnerable samples, while CWE-22 accounts for only 0.33%. This imbalanced distribution makes it difficult for the model to adequately learn the vulnerability patterns of the minority samples, leading to incorrect predictions for these samples. As shown in Fig. 12, the examples of CWE-704 demonstrate issues with incorrect type conversion transformations. First, we observe that the vulnerability pattern is complex, with a large number of program elements being involved in the bug fix. Furthermore, vulnerabilities with the type CWE-704 account for only 0.22% in the Big-Vul dataset, making the vulnerability pattern of this instance challenging for the model to mine. Therefore, these vulnerabilities are misclassified by the model due to their scarce sample numbers and the uneven distribution of data. Our future work will explore how to generate more vulnerability samples within datasets and introduce resampling techniques during the training phase to further aid the model in enhancing its performance with imbalanced samples.

7.4 Resource Consumption

When exploring the performance of a model, resource consumption is an indispensable factor to consider. Resource consumption mainly includes two aspects: model size and time consumption, both of which are related to the versatility. In this section, we will further analyze the efficiency of ANGEL by comparing its performance with baseline models in these two aspects. Model size refers to the total number of parameters in the whole model. Time consumption refers to the time spent by the model to infer a test set.

It is important to highlight that in this study, all the baselines are reproduced on our server, which means their running environment is identical to that of ANGEL, as detailed in Section 5.3. This

practice guarantees a fair comparison across various approaches in terms of time consumption.

According to Table 11, we can have the following observations:

- ANGEL has the least amount of parameters in the model, which is because the model architecture we designed follows the principle of “narrow while deep” [87], i.e., the hidden dimension is small while the model is deep. Typically, existing methods [26], [30], [37] set the hidden layer dimension to 200, while ANGEL has a hidden layer dimension of only 64. Our preliminary experiments show that directly increasing the hidden layer dimension of ANGEL does not result in a significant performance gain in the results. Therefore, our approach is more lightweight compared to existing methods.
- There are some non-parameter computing operations in ANGEL, such as APPNP and matrix multiplication in the attention mechanism. Therefore, although ANGEL has the least amount of parameters, it is at a medium level in terms of time spent.
- Although Devign and Reveal have advantages in terms of time efficiency, they rely on stacked model parameters, and their overly simplified model design fails to effectively capture vulnerability patterns.
- Overall, although ANGEL has not fully surpassed all baseline methods in terms of efficiency, it achieved the best performance in vulnerability detection tasks. Notably, ANGEL’s efficiency remains competitive. Compared to AMPLE, the baseline method with the best detection performance, ANGEL demonstrates superior efficiency and detection results, highlighting its effectiveness and practicality in vulnerability detection tasks.

Such results indicate that ANGEL is not only effective in terms of the vulnerability detection task but also lightweight with respect to the model size, which makes it easy to be deployed and executed.

7.5 Threats to Validity

7.5.1 External Threats

All our results and findings are limited to three publicly available benchmark datasets. However, these datasets were created with the C/C++ language. Other programming languages such as Python and Java have not yet been implemented. In principle, ANGEL can be applicable to other programming languages, as the method does not rely on specific artificial rules for its implementation. In the future, we will attempt to select a greater variety of programming language datasets to evaluate our approach.

7.5.2 Internal Threats

During the data pre-processing phase and the model training phase, there are some uncertainties. Specifically, we may be

unable to precisely standardize the code in the data pre-processing phase, and we use the tool Joern to parse the source code. To our knowledge, Joern is frequently updated. Different versions of Joern have slight differences in the parsing of source code, which leads to uncertainties in the input for the model. Additionally, during the model training phase, we rely on random grid search to determine the optimal hyper-parameters as best as we can, but it is impossible to try all combinations.

8 CONCLUSION

In this paper, we reveal the bottleneck of current vulnerability detectors that they usually fail to predict accurately on large code graphs. We further propose ANGEL, a hierarchical and context-aware graph representation learning approach for vulnerability detection. ANGEL is designed to (1) hierarchically reduce the size of the graph to alleviate the interference of noise information, and (2) utilize the complementarity of GT and GNN to understand the semantics of the code graph from both the global and local perspectives. Experimental results demonstrate that ANGEL is effective in vulnerability detection, achieving higher performance compared to state-of-the-art methods and mitigating the ineffectiveness of existing vulnerability detection techniques on large code graphs.

In our future work, we plan to focus on more fine-grained vulnerability detection to provide direct support for security audits and code repair. Specifically, this will include: (1) Vulnerability Type Identification. Expanding the current binary classification task into multi-label classification to enhance the interpretability of detection results; (2) Vulnerable Localization. Ranking code lines based on their suspicion levels to achieve precise identification of specific vulnerability lines.

REFERENCES

- [1] S. M. Ghaffarian and H. R. Shahriari, "Software vulnerability analysis and discovery using machine-learning and data-mining techniques: A survey," *ACM computing surveys (CSUR)*, vol. 50, no. 4, pp. 1–36, 2017.
- [2] Q. Chen and R. A. Bridges, "Automated behavioral analysis of malware: A case study of wannacry ransomware," *2017 16th IEEE International Conference on machine learning and applications (ICMLA)*, pp. 454–460, 2017.
- [3] G. Lin, S. Wen, Q.-L. Han, J. Zhang, and Y. Xiang, "Software vulnerability detection using deep neural networks: a survey," *Proceedings of the IEEE*, vol. 108, no. 10, pp. 1825–1848, 2020.
- [4] W. Wang, J. Song, G. Xu, Y. Li, H. Wang, and C. Su, "Contractward: Automated vulnerability detection models for ethereum smart contracts," *IEEE Transactions on Network Science and Engineering*, vol. 8, no. 2, pp. 1133–1144, 2020.
- [5] Z. Li, D. Zou, S. Xu, Z. Chen, Y. Zhu, and H. Jin, "Vuldelocator: a deep learning-based fine-grained vulnerability detector," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 4, pp. 2821–2837, 2021.
- [6] Y. Wu, D. Zou, S. Dou, W. Yang, D. Xu, and H. Jin, "Vulcnn: An image-inspired scalable vulnerability detection system," *Proceedings of the 44th International Conference on Software Engineering*, pp. 2365–2376, 2022.
- [7] B. Lin, S. Wang, Z. Liu, Y. Liu, X. Xia, and X. Mao, "Cct5: A code-change-oriented pre-trained model," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2023. ACM, 2023, p. 1509–1521.
- [8] Z. Zhang, Y. Lei, M. Yan, Y. Yu, J. Chen, S. Wang, and X. Mao, "Reentrancy vulnerability detection and localization: A deep learning based two-phase approach," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2023.
- [9] H. Wu, Z. Zhang, S. Wang, Y. Lei, B. Lin, Y. Qin, H. Zhang, and X. Mao, "Peculiar: Smart contract vulnerability detection based on crucial data flow graph and pre-training techniques," in *2021 IEEE 32nd International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 2021, pp. 378–389.
- [10] H. Shahriar and M. Zulkernine, "Mitigating program security vulnerabilities: Approaches and challenges," *ACM Computing Surveys (CSUR)*, vol. 44, no. 3, pp. 1–46, 2012.
- [11] S. Cherem, L. Princehouse, and R. Rugina, "Practical memory leak detection using guarded value-flow analysis," in *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2007, pp. 480–491.
- [12] T. Xie, N. Tillmann, J. De Halleux, and W. Schulte, "Fitness-guided path exploration in dynamic symbolic execution," in *2009 IEEE/IFIP International Conference on Dependable Systems & Networks*. IEEE, 2009, pp. 359–368.
- [13] D. A. Ramos and D. Engler, "{Under-Constrained} symbolic execution: Correctness checking for real code," in *24th USENIX Security Symposium (USENIX Security 15)*, 2015, pp. 49–64.
- [14] D. Zou, S. Wang, S. Xu, Z. Li, and H. Jin, "vuldeepecker: A deep learning-based system for multiclass vulnerability detection," *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 5, pp. 2224–2236, 2019.
- [15] Y. Zhuang, Z. Liu, P. Qian, Q. Liu, X. Wang, and Q. He, "Smart contract vulnerability detection using graph neural networks," in *Proceedings of the Twenty-Ninth International Conference on International Joint Conferences on Artificial Intelligence*, 2021, pp. 3283–3290.
- [16] S. Wang, M. Wen, B. Lin, Y. Liu, T. F. Bissyandé, and X. Mao, "Pre-implementation method name prediction for object-oriented programming," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 6, pp. 1–35, 2023.
- [17] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, and Y. Zhong, "Vuldeepecker: A deep learning-based system for vulnerability detection," *arXiv preprint arXiv:1801.01681*, 2018.
- [18] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, and Z. Chen, "Sysevr: A framework for using deep learning to detect software vulnerabilities," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 4, pp. 2244–2258, 2021.
- [19] C. Thapa, S. I. Jang, M. E. Ahmed, S. Camtepe, J. Pieprzyk, and S. Nepal, "Transformer-based language models for software vulnerability detection," in *Proceedings of the 38th Annual Computer Security Applications Conference*, 2022, pp. 481–496.
- [20] S. Cao, X. Sun, L. Bo, Y. Wei, and B. Li, "Bgnn4vd: Constructing bidirectional graph neural-network for vulnerability detection," *Information and Software Technology*, vol. 136, p. 106576, 2021.
- [21] G. Ye, Z. Tang, H. Wang, D. Fang, J. Fang, S. Huang, and Z. Wang, "Deep program structure modeling through multi-relational graph-based learning," in *Proceedings of the ACM International conference on parallel architectures and compilation techniques*, 2020, pp. 111–123.
- [22] Z. Zhen, X. Zhao, J. Zhang, Y. Wang, and H. Chen, "Da-gnn: A smart contract vulnerability detection method based on dual attention graph neural network," *Computer Networks*, p. 110238, 2024.
- [23] S. Cao, X. Sun, X. Wu, D. Lo, L. Bo, B. Li, and W. Liu, "Coca: Improving and explaining graph neural network-based vulnerability detection systems," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.
- [24] C. Zhang, B. Liu, Y. Xin, and L. Yao, "Cpvd: Cross project vulnerability detection based on graph attention network and domain adaptation," *IEEE Transactions on Software Engineering*, 2023.
- [25] V.-A. Nguyen, D. Q. Nguyen, V. Nguyen, T. Le, Q. H. Tran, and D. Phung, "Regvd: Revisiting graph neural networks for vulnerability detection," in *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*, 2022, pp. 178–182.
- [26] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," *Advances in neural information processing systems*, vol. 32, 2019.
- [27] Y. Li, D. Tarlow, M. Brockschmidt, and R. Zemel, "Gated graph sequence neural networks," *arXiv preprint arXiv:1511.05493*, 2015.
- [28] F. Luo, R. Luo, T. Chen, A. Qiao, Z. He, S. Song, Y. Jiang, and S. Li, "Scvhunter: Smart contract vulnerability detection based on heterogeneous graph attention network," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.
- [29] X.-C. Wen, C. Gao, J. Ye, Y. Li, Z. Tian, Y. Jia, and X. Wang, "Meta-path based attentional graph learning model for vulnerability detection," *IEEE Transactions on Software Engineering*, 2023.
- [30] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep learning based vulnerability detection: Are we there yet?" *IEEE Transactions on Software Engineering*, vol. 48, no. 9, pp. 3280–3296, 2021.
- [31] J. Fan, Y. Li, S. Wang, and T. N. Nguyen, "Ac/c++ code vulnerability dataset with code changes and cve summaries," in *Proceedings of the*

- 17th International Conference on Mining Software Repositories, 2020, pp. 508–512.
- [32] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, “Modeling and discovering vulnerabilities with code property graphs,” in *2014 IEEE Symposium on Security and Privacy*. IEEE, 2014, pp. 590–604.
- [33] G. Yehudai, E. Fetaya, E. Meirum, G. Chechik, and H. Maron, “From local structures to size generalization in graph neural networks,” *International Conference on Machine Learning*, pp. 11 975–11 986, 2021.
- [34] Z. Liu, Q. Mao, C. Liu, Y. Fang, and J. Sun, “On size-oriented long-tailed graph classification of graph neural networks,” *Proceedings of the ACM Web Conference*, pp. 1506–1516, 2022.
- [35] S. Maskey, R. Levie, Y. Lee, and G. Kutyniok, “Generalization analysis of message passing neural networks on large random graphs,” *Advances in neural information processing systems*, vol. 35, pp. 4805–4817, 2022.
- [36] X. Chu, Y. Jin, X. Wang, S. Zhang, Y. Wang, W. Zhu, and H. Mei, “Wasserstein barycenter matching for graph size generalization of message passing neural networks,” *International Conference on Machine Learning*, pp. 6158–6184, 2023.
- [37] X.-C. Wen, Y. Chen, C. Gao, H. Zhang, J. M. Zhang, and Q. Liao, “Vulnerability detection with graph simplification and enhanced graph representation learning,” *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pp. 2275–2286, 2023.
- [38] Y. Li, S. Wang, and T. N. Nguyen, “Vulnerability detection with fine-grained interpretations,” *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pp. 292–303, 2021.
- [39] X. Duan, J. Wu, S. Ji, Z. Rui, T. Luo, M. Yang, and Y. Wu, “Vulsniper: focus your attention to shoot fine-grained vulnerabilities,” in *Proceedings of the 28th International Joint Conference on Artificial Intelligence*, 2019, pp. 4665–4671.
- [40] D. Hin, A. Kan, H. Chen, and M. A. Babar, “Linevd: Statement-level vulnerability detection using graph neural networks,” *Proceedings of the 19th international conference on mining software repositories*, pp. 596–607, 2022.
- [41] X. Cheng, H. Wang, J. Hua, G. Xu, and Y. Sui, “Deepwukong: Statically detecting software vulnerabilities using deep graph neural network,” *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 30, no. 3, pp. 1–33, 2021.
- [42] U. Alon and E. Yahav, “On the bottleneck of graph neural networks and its practical implications,” in *Proceedings of the 9th International Conference on Learning Representations*, 2021.
- [43] H. Gao and S. Ji, “Graph u-nets,” in *international conference on machine learning*. PMLR, 2019, pp. 2083–2092.
- [44] D. Chen, L. O’Bray, and K. Borgwardt, “Structure-aware transformer for graph representation learning,” in *International Conference on Machine Learning*. PMLR, 2022, pp. 3469–3489.
- [45] L. Rampásek, M. Galkin, V. P. Dwivedi, A. T. Luu, G. Wolf, and D. Beaini, “Recipe for a general, powerful, scalable graph transformer,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 14 501–14 515, 2022.
- [46] S. Wang, M. Geng, B. Lin, Z. Sun, M. Wen, Y. Liu, L. Li, T. F. Bissyandé, and X. Mao, “Fusing code searchers,” *IEEE Transactions on Software Engineering*, 2024.
- [47] S. Wang, B. Lin, Z. Sun, M. Wen, Y. Liu, Y. Lei, and X. Mao, “Two birds with one stone: Boosting code generation and code search via a generative adversarial network,” *Proceedings of the ACM on Programming Languages*, vol. 7, no. OOPSLA2, pp. 486–515, 2023.
- [48] S. Liu, “A unified framework to learn program semantics with graph neural networks,” in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*, 2020, pp. 1364–1366.
- [49] Y. Hu, D. Zou, J. Peng, Y. Wu, J. Shan, and H. Jin, “Treecen: Building tree graph for scalable semantic code clone detection,” in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–12.
- [50] J. Yang, O. Ruan, and J. Zhang, “Tensor-based gated graph neural network for automatic vulnerability detection in source code,” *Software Testing, Verification and Reliability*, p. e1867, 2024.
- [51] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and S. Y. Philip, “A comprehensive survey on graph neural networks,” *IEEE transactions on neural networks and learning systems*, vol. 32, no. 1, pp. 4–24, 2020.
- [52] X. Peng, J. Cheng, X. Tang, J. Liu, and J. Wu, “Dual contrastive learning network for graph clustering,” *IEEE Transactions on Neural Networks and Learning Systems*, 2023.
- [53] X. Peng, J. Cheng, X. Tang, B. Zhang, and W. Tu, “Multi-view graph imputation network,” *Information Fusion*, vol. 102, p. 102024, 2024.
- [54] Q. Wu, W. Zhao, C. Yang, H. Zhang, F. Nie, H. Jiang, Y. Bian, and J. Yan, “Simplifying and empowering transformers for large-graph representations,” *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [55] E. Ranjan, S. Sanyal, and P. Talukdar, “Asap: Adaptive structure aware pooling for learning hierarchical graph representations,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, no. 04, 2020, pp. 5470–5477.
- [56] C. Liu, Y. Zhan, J. Wu, C. Li, B. Du, W. Hu, T. Liu, and D. Tao, “Graph pooling for graph neural networks: progress, challenges, and opportunities,” in *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence*, 2023, pp. 6712–6722.
- [57] Z. Ying, J. You, C. Morris, X. Ren, W. Hamilton, and J. Leskovec, “Hierarchical graph representation learning with differentiable pooling,” *Advances in neural information processing systems*, vol. 31, 2018.
- [58] M. Li, S. Chen, Y. Zhang, and I. Tsang, “Graph cross networks with vertex infomax pooling,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 14 093–14 105, 2020.
- [59] B. Jiang, Y. Liu, and W. K. Chan, “Contractfuzzer: Fuzzing smart contracts for vulnerability detection,” in *Proceedings of the 33rd ACM/IEEE international conference on automated software engineering*, 2018, pp. 259–269.
- [60] Z. Song, J. Wang, K. Yang, and J. Wang, “Hgivul: Detecting inter-procedural vulnerabilities based on hypergraph convolution,” *Information and Software Technology*, vol. 160, p. 107219, 2023.
- [61] H. Sun, L. Cui, L. Li, Z. Ding, S. Li, Z. Hao, and H. Zhu, “Vdtriple: Vulnerability detection with graph semantics using triplet model,” *Computers & Security*, vol. 139, p. 103732, 2024.
- [62] A. Sejjia, S. Das, S. Shafiq, and N. Medvidović, “Toward improved deep learning-based vulnerability detection,” in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 2024, pp. 1–12.
- [63] G. Lin, J. Zhang, W. Luo, L. Pan, O. De Vel, P. Montague, and Y. Xiang, “Software vulnerability discovery via learning multi-domain knowledge bases,” *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 5, pp. 2469–2485, 2019.
- [64] V. Nguyen, T. Le, T. Le, K. Nguyen, O. DeVel, P. Montague, L. Qu, and D. Phung, “Deep domain adaptation for vulnerable code function identification,” in *2019 international joint conference on neural networks (IJCNN)*. IEEE, 2019, pp. 1–8.
- [65] S. Liu, G. Lin, L. Qu, J. Zhang, O. De Vel, P. Montague, and Y. Xiang, “Cd-vuld: Cross-domain vulnerability discovery based on deep domain adaptation,” *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 1, pp. 438–451, 2020.
- [66] B. Steenhoek, H. Gao, and W. Le, “Dataflow analysis-inspired deep learning for efficient vulnerability detection,” in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 2024, pp. 1–13.
- [67] W. Zheng, Y. Jiang, and X. Su, “Vulspg: Vulnerability detection based on slice property graph representation learning,” in *2021 IEEE 32nd International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 2021, pp. 457–467.
- [68] S. Cao, X. Sun, X. Wu, D. Lo, L. Bo, B. Li, X. Liu, X. Lin, and W. Liu, “Snopy: Bridging sample denoising with causal graph learning for effective vulnerability detection,” in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 606–618.
- [69] T. Mikolov, K. Chen, G. Corrado, and J. Dean, “Efficient estimation of word representations in vector space,” *arXiv preprint arXiv:1301.3781*, 2013.
- [70] J. Gasteiger, A. Bojchevski, and S. Günnemann, “Predict then propagate: Graph neural networks meet personalized pagerank,” in *International Conference on Learning Representations*, 2018.
- [71] J. Chen, Y. Yin, S. Cai, W. Wang, S. Wang, and J. Chen, “ignnvd: a novel software vulnerability detection model based on integrated graph neural networks,” *Science of Computer Programming*, vol. 238, p. 103156, 2024.
- [72] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer normalization,” *arXiv preprint arXiv:1607.06450*, 2016.
- [73] S. Lipp, S. Banescu, and A. Pretschner, “An empirical study on the effectiveness of static c code analyzers for vulnerability detection,” in *Proceedings of the 31st ACM SIGSOFT international symposium on software testing and analysis*, 2022, pp. 544–555.
- [74] R. K. McLean, “Comparing static security analysis tools using open source software,” in *2012 IEEE Sixth International Conference on Software Security and Reliability Companion*. IEEE, 2012, pp. 68–74.
- [75] J. D. Pereira and M. Vieira, “On the use of open-source c/c++ static analysis tools in large projects,” in *2020 16th European Dependable Computing Conference (EDCC)*. IEEE, 2020, pp. 97–102.

- [76] F. Qiu, Z. Liu, X. Hu, X. Xia, G. Chen, and X. Wang, "Vulnerability detection via multiple-graph-based code representation," *IEEE Transactions on Software Engineering*, 2024.
- [77] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations*, 2018.
- [78] L. Van der Maaten and G. Hinton, "Visualizing data using t-sne," *Journal of machine learning research*, vol. 9, no. 11, 2008.
- [79] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proceedings of the 5th International Conference on Learning Representations*, 2017.
- [80] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proceedings of the 6th International Conference on Learning Representations*, 2018.
- [81] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.
- [82] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan *et al.*, "Deepseek-v3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.
- [83] A. Hurst, A. Lerer, A. P. Goucher, A. Perelman, A. Ramesh, A. Clark, A. Ostrow, A. Welihinda, A. Hayes, A. Radford *et al.*, "Gpt-4o system card," *arXiv preprint arXiv:2410.21276*, 2024.
- [84] A. Zibaeirad and M. Vieira, "Reasoning with llms for zero-shot vulnerability detection," *arXiv preprint arXiv:2503.17885*, 2025.
- [85] Y. Ding, Y. Fu, O. Ibrahim, C. Sitawarin, X. Chen, B. Alomair, D. Wagner, B. Ray, and Y. Chen, "Vulnerability detection with code language models: How far are we?" in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 2024, pp. 469–481.
- [86] Z. Gao, H. Wang, Y. Zhou, W. Zhu, and C. Zhang, "How far have we gone in vulnerability detection using large language models," *arXiv preprint arXiv:2311.12420*, 2023.
- [87] M. Raghu, B. Poole, J. Kleinberg, S. Ganguli, and J. Sohl-Dickstein, "On the expressive power of deep neural networks," in *international conference on machine learning*. PMLR, 2017, pp. 2847–2854.